

Entwicklung eines Plug-In Electric Vehicle (PEV) Simulators zur Prüfung von Ladesäulen (EVSE)

Gregor Sellschopp,

21.09.2026

Abstract

The Combined Charging System (CCS) is the dominant standard for DC fast charging of electric vehicles in Europe, combining analog Control Pilot signaling with Power Line Communication for high-level protocol exchange.

Testing charging infrastructure typically requires a physical vehicle. Existing open-source implementations focus on the charging station side and leave vehicle-side emulation aside.

This thesis develops an integrated PEV simulator based on the EVERest framework, covering the full CCS charging sequence from IEC 61851 and SLAC through ISO 15118-2. A custom interface PCB handles low-level control via a microcontroller, while a custom EVERest module bridges hardware and protocol stack. A deployable Yocto image for the Raspberry Pi is produced to have a portable and reproducible setup.

Both AC and DC charging sessions are successfully demonstrated. EVERest proves flexible and capable, though complex to adopt. Open points include a heartbeat mechanism for safe shutdown and support for higher-current contactors.

Inhaltsverzeichnis

Abkürzungsverzeichnis	XI
Abbildungsverzeichnis	XIII
Tabellenverzeichnis.....	XV
1 Einleitung	1
2 Stand der Technik	3
2.1 DC-Schnellladen mit CCS	3
2.1.1 Architektur und Steckernorm	3
2.1.2 Control Pilot und Ladeablauf.....	5
2.1.3 Einordnung in OSI-Modell.....	6
2.1.4 Vergleich mit anderen Standards.....	7
2.2 Bestehende CCS Implementierungen	8
2.2.1 pyPLC / foccci	8
2.2.2 Everest	8
3 Methode.....	13
3.1 Anforderungen.....	13
3.1.1 Funktional	13
3.1.2 Nicht-funktional	13
3.2 Systemarchitektur	14
3.2.1 Hardware	14
3.2.2 Software.....	15
3.3 Bauteilauswahl und -auslegung.....	16
3.3.1 CCS Stecker	16
3.3.2 CP Messung und Steuerung.....	17
3.3.3 Leistungsmessung	18
3.3.4 Leistungstrenner und Relais	21
3.3.5 SBC und Prozessor	22
3.3.6 PCB	22
3.4 Dieter-Firmware	24
3.4.1 Befehle.....	25
3.4.2 Messwerte	26

3.5	Everest	26
3.5.1	Das ev_board_support Interface.....	27
3.5.2	DieterEvDriver Modul.....	29
3.5.3	Laufzeitkonfiguration.....	31
3.6	Build- und Entwicklungswerkzeuge	31
3.6.1	CMake	31
3.6.2	Yocto.....	32
3.6.3	Open-plc-utils.....	34
3.6.4	Venv & ev-cli.....	35
3.6.5	Admin panel	36
3.6.6	MQTT Explorer	36
3.6.7	Wireshark.....	36
3.6.8	PlatformIO.....	37
4	Ergebnisse	39
5	Zusammenfassung, Schlussfolgerungen und Ausblick	43
6	Literatur	44
Anhang		49

Abkürzungsverzeichnis

AC	Alternating Current
ADC	Analog Digital Converter
BSP	Board Support Package
CCS	Combined Charging System
CLI	Command Line Interface
CMD	Command
CP	Control Pilot
DC	Direct Current
DHCP	Dynamic Host Configuration Protocol
DIETER	Device Interface ElecTronic Especially not limited to Raspberry
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
GB/T	National Standard Recommended (chinesisches Äquivalent)
HLC	High Level Communication
HV	High Voltage
LDO	Low Dropout Regulator
LFE	Linux Foundation Energy
MCU	Micro Controller Unit
MOSFET	Metal Oxides Semiconductor Field Effect Transistor
MQTT	Message Queuing Telemetry Transport
NACS	North American Charging Standard
OCPP	Open Charge Point Protocol
OSI	Open System Interconnection model
PE	Protective Earth
PEV	Plug-in Electric Vehicle
PLC	Power Line Communication
PP	Proximity Pilot
PV	Photovoltaik
PWM	Pulsweitenmodulation
SBC	Single Board Computer
SLAC	Signal Level Attenuation Characterization

UART	Universal Asynchronous Receiver-Transmitter
VAR	Variable
V2G	Vehicle To Grid

Abbildungsverzeichnis

Abbildung 2.1:	Typ 2 Combo Stecker (fahrzeugseitig) [1]	3
Abbildung 2.2:	Typ 1 Combo Stecker [2]	4
Abbildung 2.3:	Widerstandsnetzwerk AC Laden [3, S. 28]	5
Abbildung 2.4:	V2G OSI [3, S. 43]	7
Abbildung 3.1:	Hardwarearchitektur, angelehnt an [22]	14
Abbildung 3.2:	Softwarearchitektur	15
Abbildung 3.3:	Schaltplan für CP Messung und Steuerung	18
Abbildung 3.4:	Schaltplan zum Verarbeiten eines Analogsignals	20
Abbildung 3.5:	Schaltplan zur Spannungsmessung	21
Abbildung 3.6:	Visualisierung des PCB	22
Abbildung 3.7:	Fehlerhafte IP Zuweisung des Homeplug Modems	37
Abbildung 4.1	Bau des Fahrzeugsimulators	40
Abbildung 4.2:	Tee kochen am Alpitronic HYC-300	41
Abbildung B.1:	Schaltplan Dieter	52
Abbildung C.1	Maße Platine	53

Tabellenverzeichnis

Tabelle 2.1:	Benennung Combo-2 Kontakte [1]	4
Tabelle 2.2:	Vergleich CCS Spezifikationen [1], [2].....	4
Tabelle 2.3:	CP Zustände [3, S. 28]	6
Tabelle 3.1:	Bidirektionale Motorsteuerung, nach [26, S. 9].....	17
Tabelle 3.2:	Funktionsbausteine PCB	23
Tabelle 3.3:	wichtige Begriffe zu Isolation [27]	23
Tabelle 3.4:	Protokoll MCU - Linux.....	24
Tabelle 3.5:	relevante Datenblattangaben des Verriegelungsaktuators [23, S. 3]	25
Tabelle 3.6:	von DieterEvDriver nicht unterstützte Befehle	29
Tabelle 3.7:	von DieterEvDriver unterstützte Befehle	29

1 Einleitung

Die Elektromobilität hat sich in den vergangenen Jahren von einer Nischentechnologie zu einem zentralen Bestandteil der Verkehrswende entwickelt. In Europa hat sich das Combined Charging System (CCS) als dominierender Standard für das DC-Schnellladen etabliert. Es ermöglicht Ladeleistungen von mehreren hundert Kilowatt und setzt dabei auf ein Zusammenspiel von mehreren Protokollen.

Die Interoperabilität zwischen Fahrzeug und Ladestation ist dabei keine Selbstverständlichkeit. Obwohl der Standard existiert, sind die tatsächlichen Implementierungen teils proprietär, intransparent und oft auch nicht normgerecht.

Für die Entwicklung und Inbetriebnahme von Ladeinfrastruktur ist es notwendig, den fahrzeugseitigen Teil des Protokolls zu emulieren. Das ermöglicht einfaches Testen der Ladesäule auf grundlegende Funktion und Sondersituationen. Bestehende Open-Source Implementierungen legen den Fokus dabei auf die Ladestationsseite.

Ziel dieser Arbeit ist die Entwicklung eines solchen PEV-Simulators. Der Emulator soll den CCS Ladevorgang vollständig abbilden, von der IEC61851 Signalisierung über das SLAC-Protokoll bis hin zur ISO15118-2 Kommunikation, und dabei in eine eigenständige Hardwareeinheit integriert sein.

Im Folgenden werden zunächst die relevanten technischen Grundlagen des CCS sowie bestehende Implementierungen vorgestellt. Kapitel 3 beschreibt die methodische Vorgehensweise, von den Anforderungen über die Hardware- und Softwarearchitektur bis hin zur Implementierung der einzelnen Komponenten. Die erzielten Ergebnisse werden in Kapitel 4 dargestellt, in Kapitel 5 zusammengefasst und einem Ausblick auf offene Aufgaben verwiesen.

2 Stand der Technik

2.1 DC-Schnellladen mit CCS

Das Combined-Charging-System (CCS) ist ein internationaler Standard für das Laden von Elektrofahrzeugen. Dieses kombiniert verschiedene Schnittstellen, die bisher unterschiedliche Anwendungen hatten: Ein- und mehrphasiges AC-Laden und DC-Schnellladen. Die Umsetzung dieser Anforderungen führt zu einem recht komplexen Ablauf des Ladevorgangs, der in diesem Kapitel grob erläutert werden soll.

2.1.1 Architektur und Steckernorm

Eine wichtige Anforderung beim Entwurf des Steckers war der Rückwärtskompatibilität mit den jeweiligen Vorgängerstandards. In Europa ist das IEC 62196 Typ 2 (auch IEC Typ 2 genannt), welcher 2013 festgelegt wurde und AC-Laden mit bis zu drei Phasen regelt. Aufgrund der weiten Verbreitung wurde dieser Stecker für CCS übernommen (festgelegt in IEC 62196-3) und mit zwei stärkeren Kontakten für das DC-Laden erweitert. Die Kombination wird als „Combo-2“ Stecker-Standard bezeichnet. Häufig werden dabei die nicht erwünschten Kontakte für das AC Laden weggelassen.

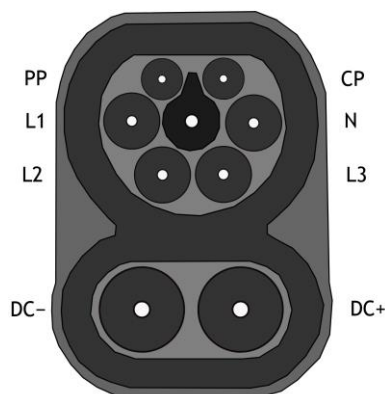


Abbildung 2.1: Typ 2 Combo Stecker (fahrzeugseitig) [1]

Tabelle 2.1: Benennung Combo-2 Kontakte [1]

Abkürzung	Beschreibung
PE	Schutzleiter
CP	Dialog zwischen Fahrzeug und Ladesäule mittels Analogsignal
PP	Begrenzung des Ladestroms mittels Widerstandscodierung
DC+	Gleichstromladung, Pluspol
DC-	Gleichstromladung, Minuspol

Es soll erwähnt sein, dass die Bezeichnungen häufig nicht korrekt verwendet werden: „CCS“ an sich bezeichnet nicht den Stecker, sondern insbesondere das Kommunikationsprotokoll. So wurde z.B. in Nordamerika ebenfalls das CCS-Protokoll übernommen, jedoch mit einem anderen Stecker. Dort wurde die Typ-1-Fahrzeugkupplung (SAE J1772) als Basis verwendet, weshalb die CCS Bauform als „Combo 1“ bekannt ist.



Abbildung 2.2: Typ 1 Combo Stecker [2]

Historisch wurden zwei Spezifikationen des CCS entwickelt, deren Kerndaten in Tabelle 2.2 dargestellt sind.

Tabelle 2.2: Vergleich CCS Spezifikationen [1], [2]

	CCS 1.0	CCS 2.0
Erscheinungsjahr	2013	2016
Spannung	max. 500 V	200 – 1000 V
Strom	max. 400 A	max. 500A
Leistung	max. 200 kW	max. 500 kW
Anschluss	Combo 1 oder 2 (IEC 62196-3)	
Kommunikation	DIN SPEC 70121	DIN SPEC 70121 ISO 15118
Lastverteilung	Reaktiv	Reaktiv und geplant
Freischaltung	Externe Zahlung	Externe Zahlung / Plug&Charge

Hieraus wird auch ersichtlich, dass die häufige Bezeichnung der Combo-2 Steckerkombination als „CCS2 Stecker“ nicht korrekt ist, weil es nicht auf die Spezifikation des physischen Steckers verweist.

Die Kommunikation lässt sich hierarchisch in zwei Teile separieren: Die sogenannte Low-level Kommunikation, welche die Logik für das AC Laden enthält, wurde von CCS übernommen und läuft über die CP und PP Kontakte. Anschließend kann das Fahrzeug der EVSE (Electric Vehicle Supply Equipment – Ladesäule) den Wechsel zum DC Laden signalisieren und so die Aushandlung der Parameter für CCS zu initiieren.

2.1.2 Control Pilot und Ladeablauf

Beim AC-Laden können die relevanten Verbindungen wie folgt dargestellt werden:

Da die Spannung beim AC-Laden bekannt ist, muss dem Fahrzeug nur mitgeteilt werden, welche maximale Leistung es abrufen darf. Zu diesem Zweck wird ein 1 kHz PWM-Signal (+/- 12 V) zwischen CP- und PE-Kontakte gelegt. Je nach zulässiger Stromstärke stellt die Ladestation den duty cycle ein. Während z.B. 60 % einen zulässigen Strom von 36 A bedeuten, muss sich das Fahrzeug bei 30 % duty cycle mit 18 A begnügen. [3, S. 27]

Wenn dagegen mit Gleichstrom geladen werden soll, wird ein duty cycle von 5 % erzeugt und damit der Wechsel zur Powerline Kommunikation (PLC) initiiert. Diese wird im Folgenden als high-level Kommunikation (HLC) bezeichnet und im Abschnitt 2.1.3 erläutert.

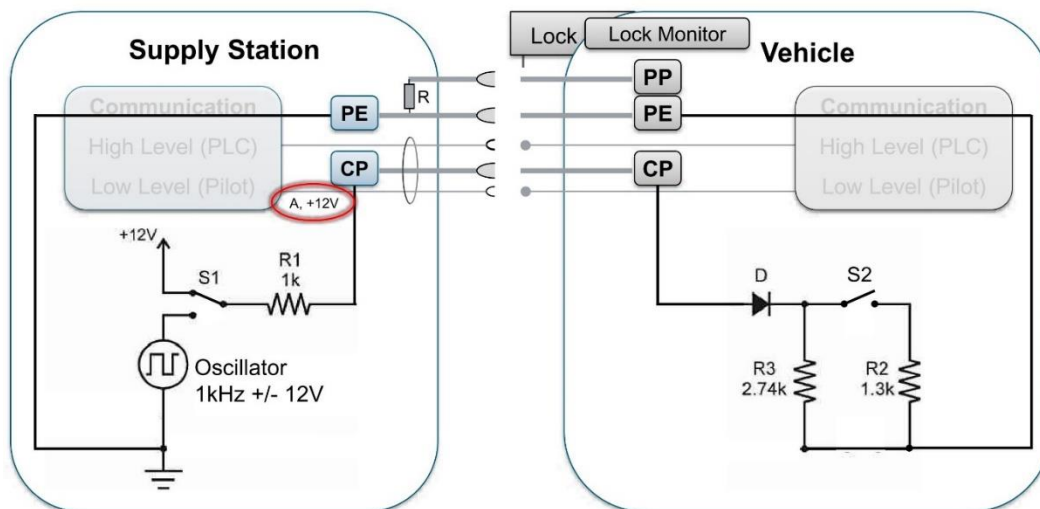


Abbildung 2.3: Widerstandsnetzwerk AC Laden [3, S. 28]

Die Ladestation kann über den Zustand des Fahrzeugs über das Spannungsniveau des CP-Kontakts schließen. Sobald ein Fahrzeug angesteckt wird, sinkt der Betrag des Spannungswerts aufgrund des gebildeten Widerstandsnetzwerks auf 9 V. Schließt das Fahrzeug zusätzlich S2, signalisiert es Bereitschaft. In IEC 61851-1 werden sechs Zustände genormt, die den Zustand des Fahrzeugs mithilfe der CP-Leitung darstellen:

Tabelle 2.3: CP Zustände [3, S. 28]

Zustand	Spannung	Beschreibung
A	+12 V	- Kein Fahrzeug angesteckt
		- Verbindung mit Fahrzeug hergestellt
B	+9 V	- Fahrzeug nicht bereit
		- Stromversorgung Ladestation aus
C	+6 V	- Fahrzeug bereit
		- Stromversorgung Ladestation an
		- Fahrzeug bereit
D	+3 V	- Stromversorgung Ladestation an
		- Belüftung erforderlich
E	0 V	- Kurzschluss zwischen CP und PE
		- Entriegeln nach max. 30 ms
F	-12 V	- Ladestation nicht bereit

2.1.3 Einordnung in OSI-Modell

Die gesamte Kommunikation basiert auf dem sieben-stufigen OSI-Modell, wobei jede dieser Schichten einen bestimmten Prozess für die hierarchisch höhere Schicht ausführt. Dies funktioniert, indem jede Schicht ihren Header vor die Nachricht schreibt.

Die Verbindung von CP und PE und die Signalisierung von 5 % PWM auf CP bilden dabei Schicht 1. Somit ist die Vorbedingung für Schicht 2 erfüllt: Die Konfiguration der PLC-Modems und Start der HLC. Ein für diesen Vorgang charakteristischer Prozess ist die sogenannte Signal Level Attenuation Characterization (SLAC). Hierbei wird anhand der Signalstärke sichergestellt, dass sich ein Fahrzeug per PLC mit der tatsächlich verbundenen Ladesäule verbindet und nicht mit dem benachbarten Gerät, da es aufgrund von elektromagnetischen Störungen zu Kopplung kommen kann.

Zwischen der physikalischen Schicht und der Applikationsschicht sind noch einige weitere Schichten notwendig, die für diese Arbeit aber nicht weiter von Relevanz sind. Unter anderem sind dies TCP und TLS für eine verschlüsselte Kommunikation, EXI für kompakte Datenübertragung und V2G Sitzungsschicht. [3, S. 40-54]

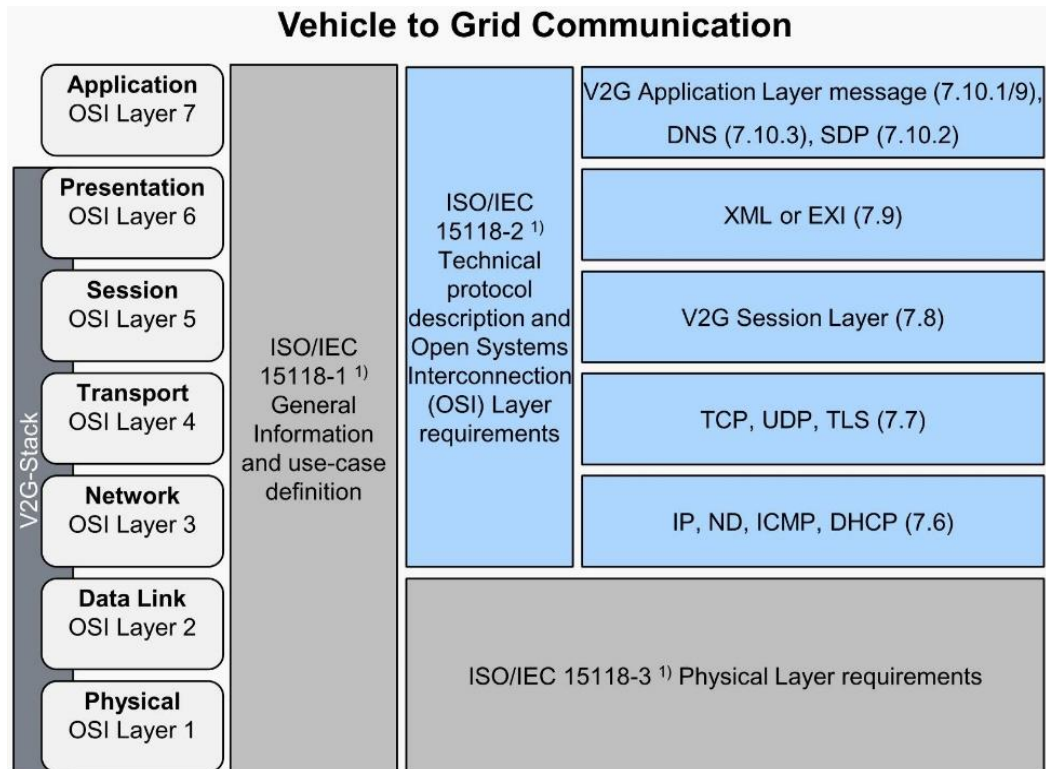


Abbildung 2.4: V2G OSI [3, S. 43]

2.1.4 Vergleich mit anderen Standards

Natürlich gibt es noch andere Möglichkeiten mit Gleichstrom Fahrzeuge zu laden. Vor der Einführung von CCS war der CHAdeMO Standard in Europa prominentester Vertreter. Dieser wurde vor allem von japanischen Herstellern vorangetrieben, jedoch hatte er einen entscheidenden Nachteil: AC-Laden ist nicht möglich, weshalb ein separater Stecker dafür notwendig ist. Andere Unterscheidungsmerkmale sind zusätzliche Kontakte für die Steuerung der Schütze im Fahrzeug und eine eigene Stromversorgung für Nebenverbraucher. [4] Obwohl die Kommunikation im Gegensatz zu CCS auf CAN basiert und dadurch wesentlich einfacher umgesetzt werden kann¹, hat sich CHAdeMO nicht durchgesetzt: von knapp 200 k Ladepunkten in Deutschland waren Stand 2026 nur etwa 3300 mit CHAdeMO Anschluss ausgerüstet. [6]

Trotzdem hat sich die Grundidee der CAN-Implementierung gehalten und ist Teil des in China dominierenden GB/T Standards. Dieser hat im Vergleich zu CCS eine wesentlich höhere Minimalspannung von 750 V und verfügt ebenfalls über ein separates Paar Kabel zur Datenübertragung sowie CP/PP Pendants. [1], [7]

Die dritte international relevante Lösung ist der von Tesla entwickelte North American Charging Standard (NACS), der aber ebenfalls das CCS Protokoll übernommen hat, jedoch mit einem eigenen, kompakteren Stecker. [8]

¹ Laut [5] ein bis zu zwanzig fach niedrigerer Implementierungsaufwand

2.2 Bestehende CCS Implementierungen

2.2.1 pyPLC / foccci

Das erste offene Projekt zur CCS Implementierung ging vom openinverter Forum Anfang 2022 aus. Hier wurde versucht mit handelsüblichen Homeplug PLC Adaptern die Kommunikation zwischen Ladestation und Fahrzeug mitzuschneiden. Nötig dafür waren das Hinzufügen von einigen elektronischen Komponenten, eine Neuprogrammierung der Modems mithilfe eines offenen Toolkits von Qualcomm Atheros und eine externe Stromversorgung. [9], [10], [11]

Github Nutzer uhi22 baute auf dieser Basis auf und schrieb die Python Software pyPLC, die sowohl auf Linux, als auch auf Microsoft Windows lauffähig ist. [11] Ziel war es, die gesamte Kommunikation abzuhören, aber auch jeweils die Fahrzeug- und Ladestationsseite zu simulieren. [12] Die Grundidee bestand darin, zwei separate Hardwarebausteine für jeweils die Low-level Kommunikation über den CP Pin und die high-level Kommunikation per Ethernet über PLC zu schaffen. Während das Homeplug Modem per Ethernet an den Computer angeschlossen war, wurde eine separate Elektronik u.a. für die Schaltung des CP MOSFETs (S2 in Abbildung 2.3: Widerstandsnetzwerk AC Laden [3, S. 28]) und die Steuerung der Schütze entwickelt. Diese basierte auf einem Arduino und kommunizierte per UART mit dem Hauptrechner (häufig ein Raspberry Pi). [11], [13]

Das Projekt war so erfolgreich, dass infolgedessen in Zusammenarbeit mit der Community auf openinverter.org ein eigenes - ebenfalls quelloffenes - Steuergerät entwickelt wurde, um Fahrzeuge mit CCS nachzurüsten. Das Fully Open-source CCS Controller Interface (FOCCCI) ist ein vollintegriertes Board, das die CP Beschaltung und ein PLC Modem, aber auch z.B. Treiber für Leistungsschütze und ein CAN Interface Richtung Fahrzeug beinhaltet². [14] [15] Die Software ist eine für Microcontroller portierte Version von pyPLC. [12] Die gesamte Entwicklung wurde 2024 auf dem Augsburg Linux Info Tag präsentiert. [5]

Trotz der insgesamt guten Dokumentation schien dieses Projekt nicht die beste Option um einen PEV Simulator zu bauen. Hauptgrund war, dass die Implementierung des CCS Protokolls doch komplex ist und die Unsicherheit bezüglich normgerechter Umsetzung groß ist. Außerdem ist der zukünftige Support entsprechend des kleinen Umfangs des Projekts fraglich. Nichtsdestotrotz stellte es einige große Menge an nützlichen Informationen bereit und wurde regelmäßig in dieser Arbeit als Referenz verwendet.

2.2.2 EVerest

Im Gegensatz zu pyPLC hat das EVerest Projekt einen wesentlich größeren Umfang. Nach der Initiierung durch die Pionix GmbH haben mittlerweile viele Privatpersonen, Universitäten und andere Firmen Code beigetragen. Auch wenn die Hauptentwicklung in Deutschland stattfindet, wird es auf der ganzen Welt zur Entwicklung von EVSE verwendet, auch durch die Vernetzung als Teil von Linux Foundation Energy (LFE). EVerest läuft geräteunabhängig und kann sowohl

² Wird u.a. im [openinverter shop](#) verkauft

für die Entwicklung von alleinstehenden AC-Ladern für den Heimgebrauch als auch komplex verzweigten öffentlichen DC-Ladesäulen verwendet werden.

Die Software unterstützt alle Standards, um jegliche Kommunikation zwischen EVs und EVSE, aber auch PV Komponenten und OCPP zu ermöglichen. Die Motivation hinter dem Projekt war die EVSE Entwicklung zu optimieren. Stand der Technik ist, dass jeder EVSE Hersteller eigene Hard- und Software entwickelt, um benutzerdefinierte Funktionalitäten mit einzubinden. Je nach Anwendungen müssen viele Schnittstellen müssen berücksichtigt werden, u.a.:

- Ladecontroller
- OCPP
- Stromzähler
- DC Netzteile
- UI Elemente: RFID/NFC Lesegeräte, Bildschirme
- Zahlungsterminals

Die vielen möglichen Kombinationen bedeuten viel Entwicklungsaufwand, obwohl die Funktionalität oft gleich ist.

Architektur

Ziel von EVERest ist es die Logik von der Hardware zu abstrahieren, um doppelte Entwicklung zu vermeiden. Die Implementierung der verschiedenen Protokolle ist in Softwaremodulen strukturiert, die über definierte Schnittstellen kommunizieren (im Folgenden Interfaces genannt). Wenn nun eine Firma mit Hardwarebausteinen A, B und C eine EVSE bauen möchte, benötigt sie nur die Implementierung der jeweiligen Interfaces für die Interaktion mit EVERest, kann die Hauptlogik aber einfach übernehmen.

Als Informationsbus wird MQTT verwendet, mit jeder Modulinstanz als eigenem Prozess. Die Kommunikation zwischen den Modulen erfolgt durch das Publishen der im Modulinterface festgelegten Variablen mit der entsprechenden topic. Ebenso kann Modul A einen Befehl in Modul B durch einen MQTT Befehl aufrufen. Diese zwei Datentypen werden in EVERest CMD („command“) und VAR („variable“) genannt. [16]

Interfaces

Die Definition einer Menge an CMDs und VARs bildet in EVERest ein Interface. Zusätzlich können standardisierte Fehler festgelegt werden. Für eine einfache hypothetische Stromversorgung könnte das Interface z.B. so aussehen: An- und Ausschalten würde als CMD implementiert sein, aufrufbar von anderen Modulen. Die gemessenen Strom- und Spannungswerte werden vom Gerät regelmäßig als VAR per MQTT gepublished. Andere Module, die diese Daten benötigen, könnten diese VAR subscriben und über die Zeit beobachten.

Jedes Interface wird über eine yaml Konfigurationsdatei definiert. Diese hat folgendes Format:
[17]

```
1. description: >-
2.   Brief description of what this interface does
3.
4. cmds:
5.   command_name:
6.     description: What this command does
7.     arguments:
8.       arg_name:
9.         description: Argument description
10.        type: string
11.   result:
12.     description: Return value description
13.     type: boolean
14.
15. vars:
16.   variable_name:
17.     description: What this variable represents
18.     type: object
19.     $ref: /some_type#/TypeName
20.
21. errors:
22.   - reference: /errors/error_category
```

Module

Module werden in EVerest wiederum über einen Satz an implementierten Interfaces definiert. Diese können entweder für andere Module bereitgestellt oder von anderen Modulen benötigt werden. Jede Implementierung hat dabei eine eindeutige ID, da ein Interface auch mehrmals vorkommen werden kann.

Als Beispiel kann ein Router betrachtet werden. Dieser implementiert nicht nur einfach das IP Protokoll, sondern hat mehrere Instanzen davon (mehrere Ethernet Ports, WLAN Antennen etc.), welche für verschiedene Anwendungszwecke verwendet werden können.

Jedes erzeugte Modul hat ebenfalls eine eindeutige ID, da auch Module beliebig oft instanziiert werden. Wenn z.B. ein Befehl per MQTT aufgerufen werden soll, müssen deshalb beide IDs berücksichtigt werden. Die MQTT topics sind jeweils

```
1. Everest/{Modul Instanz ID}/{Interface Implementierung ID}/cmd
2. Everest/{Modul Instanz ID}/{Interface Implementierung ID}/var
```

Genau wie Interfaces sind auch Module durch eine Datei (manifest.yaml) definiert. Mit dieser werden hauptsächlich zwei Dinge festgelegt: Welche Interfaces ein Modul bereitstellt und benötigt sowie mögliche Konfigurationswerte (diese werden in der Laufzeitkonfiguration festgelegt, siehe Abschnitt Laufzeitkonfiguration)

Die Manifest.yaml Datei hat dabei folgendes Format: [18], [19]

```
1. description: Module description
2. config:
3.   parameter_name:
4.     description: Parameter description
5.     type: string|integer|boolean|number
6.     default: value
7. provides:
8.   interface_name:
9.     interface: interface_type
10.    description: What this interface provides
11. requires:
12.   dependency_name:
13.     interface: interface_type
14.     min_connections: 0
15.     max_connections: 1
16. metadata:
17.   license: License URL
18.   authors:
19.     - Author name
```

Everest stellt viele Module fertig zur Verfügung. Diese enthalten häufig benötigte Funktionalität, wie etwa die Kommunikation mit einem Stromzähler. In dieser Arbeit werden Module, welche mit Hardware interagieren (also das board_support Interface implementieren) von größerer Wichtigkeit sein. Für kommerzielle Hardware existieren diese bereits, wie z.B. PhyVersoBSP für den Laderegler von Phytex. [20] In dieser Arbeit wird ein Everest Modul entwickelt, welches ebenfalls das ev_board_support Interface implementiert und

Laufzeitkonfiguration

Die Laufzeitkonfiguration ist eine weitere yaml Datei, welche die Struktur der im build befindlichen Everest Instanz festlegt. Für die zu erzeugenden Module werden folgende Dinge festgelegt:

- Ihre Objekt IDs
- Welches Modul sie implementieren
- Konfigurationswerte
- Je Interface Anforderung des instanziierten Moduls:
 - o Objekt ID der Modul Instanz, welches das Interface bereitstellt
 - o ID der Interface Implementierung innerhalb des bereitstellenden Moduls

Eine beispielhafte Konfiguration ist in Anhang A zu finden.

Relevante Module für EV-Seite

Prinzipiell ist EVerest eher für die EVSE Seite ausgelegt. Der EvseManager ist hier das zentrale Modul und verwaltet Tokens, Leistungsmesser und Module für den Hardware Support. Das fahrzeugseitige Pendant ist der EvManager, welcher ebenfalls ein Interface für den Hardware Support, aber auch das fahrzeugseitige SLAC Protokoll und die ISO15118 konforme Kommunikation mit einem Fahrzeug bereitstellt. Für die Anbindung kommen folgende Module infrage:

- EvSlac: Modul von EVerest für die fahrzeugseitige SLAC Kommunikation
- pyEvJosev: Python Implementierung eines ISO15118 konformen Fahrzeugs mit-hilfe des Josev Projekts
- Store: Modul von EVerest zum Speichern von Daten im {key}:{value} Format

Zusätzlich wird je nach Hardware ein Modul benötigt, was das Interface „ev_board_support“ bereitstellt. Dessen Entwicklung ist ein relevanter Teil dieser Arbeit. [21]

3 Methode

3.1 Anforderungen

An den PEV Simulator werden folgende direkte Anforderungen hinsichtlich Funktionalität gestellt. Diese haben Implikationen für sowohl Hardware als auch Software. Weitere Anforderungen ergeben sich aus der Anwendung und Kriterien wie Sicherheit, Fehlertoleranz und Nutzbarkeit.

Nicht zuletzt sind die Gesamtkosten des Systems ebenfalls relevant. Diese Kriterien mit Verfügbarkeit, Lieferzeit und Qualität der Datenblätter abzuwägen ist nicht immer einfach.

3.1.1 Funktional

Offensichtlich soll der Aufbau eine funktionsfähige CCS Schnittstelle implementieren, um mit einer EVSE zu kommunizieren und elektrische Leistung abzurufen. Da (zumindest im ersten Schritt) keine Batterie angeschlossen sein wird, benötigt es eine künstliche Last, die extern bereitgestellt wird. Dafür braucht es geeignete Hardware, um diese Last an die Hochspannung zu schalten oder davon zu trennen. Prinzipiell geht es nicht darum, die volle Leistungsfähigkeit der Ladesäule zu testen, sondern „nur“ um die Kommunikation. Deshalb soll der Aufbau im ersten Schritt auf niedrige Leistungswerte (ca. 10 A, 400 V) ausgelegt werden, jedoch mit Möglichkeit der Erweiterung auf höhere Spannungen (CCS spezifiziert bis zu 1000 VDC).

Für den HLC Teil von CCS wird eine PLC Schnittstelle benötigt, während die low-level Kommunikation über Manipulation der CP Leitung stattfindet. Auch hierfür benötigt der Simulator geeignete Hardware.

Zusätzlich soll die Leistung gemessen werden, die die künstliche Last aufnimmt, um Angaben der EVSE zu verifizieren, aber auch um die Hardware nicht zu überlasten. Die Messwerte sollen über eine geeignete Schnittstelle zur einfachen Weiterverarbeitung zugänglich sein.

Um das Verhalten eines EVs real nachzubilden und ein möglichst hohes Maß an Sicherheit zu gewährleisten, soll der Stecker des Ladekabels „fahrzeugseitig“ verriegelt werden können. Dies ist typischerweise in Form eines Getriebemotors realisiert, der einen Stift geradlinig in eine dafür vorgesehene Aussparung im Stecker schiebt und somit verriegelt.

3.1.2 Nicht-funktional

Zusätzlich existieren noch einige weitere Anforderungen, die keinen direkten Einfluss auf die Funktionalität haben. Im Sinne eines Forschungsprojekts soll der Aufbau auf Open-Source Software basieren, welche nach Möglichkeit eine breite Basis und gute zukünftige Unterstützung bietet. Die EVerest Plattform bietet sich hierfür sehr an.

Um nicht zu sehr von spezifischer Hardware abhängig zu sein, soll die Software möglichst abstrahiert von der Hardware funktionieren. Auch in dieser Hinsicht punktet EVerest.

Die elektrische Sicherheit darf ebenso nicht vernachlässigt werden. Speziell bei den hohen Spannungen, die bei CCS zulässig sind, müssen ausreichende Kriechstrecken, galvanische Isolation wo benötigt, und natürlich Berührungsschutz eingeplant werden.

Außerdem soll die Erweiterbarkeit um AC-Funktionalität eine Möglichkeit bleiben. Wie in Abschnitt 2.1.1 gezeigt, sind dafür zusätzliche Kabel und Schütze nötig.

3.2 Systemarchitektur

3.2.1 Hardware

Für das DC Laden wird eine Elektronik benötigt, die direkt an die CCS Datenleitungen angeschlossen ist, sowie über eine geeignete Schnittstelle mit dem Everest Host kommuniziert. Das Ziel der Umsetzung ist nicht ein vollintegriertes System, sondern ein brauchbarer Forschungsaufbau, weshalb einige fertige Bausteine verwendet werden. Dazu gehört das Homeplug Modem (TPlink TL-PA4010P v5.0), welches einen PLC Controller samt Zubehör enthält. Die relevanten Schnittstellen sind ein Ethernet Port, welcher mit dem SBC verbunden wird, sowie der CP Kontakt, auf den die HLC moduliert wird.

Zusätzlich wird ein externer Motorcontroller verwendet, um den Verriegelungsmechanismus im CCS Stecker zu betätigen. Die Schütze, mit der die Hochspannung geschaltet wird, sind ebenfalls ein externes Bauelement.

Alle übrigen Komponenten (Schaltung zum Steuern und Auslesen des CP Signals, Leistungsmesser) werden in einen Aufbau integriert, welcher im folgenden „Dieter“ (Device Interface ElecTronic Especially not limited to Raspberry) genannt wird. Ehre, wem Ehre gebührt: Uwe Hennig (github.com/uhi22) erschuf diesen Namen mit dem PyPLC Projekt. [11], [13] Da seine Dokumentation sehr nützlich und maßgebliche Grundlage dieses Projekt war, wurde der Name übernommen.

Die Bauteile, welche mit der Hochspannung zu tun haben, werden entsprechend der Reihenfolge im CCS Design Guide [3, S. 11] verbaut.

Eine Übersicht der Hardware ist in Abbildung 3.1 dargestellt, die Isolationsgrenze zwischen Mikrocontroller und HV System ist in grün gekennzeichnet.

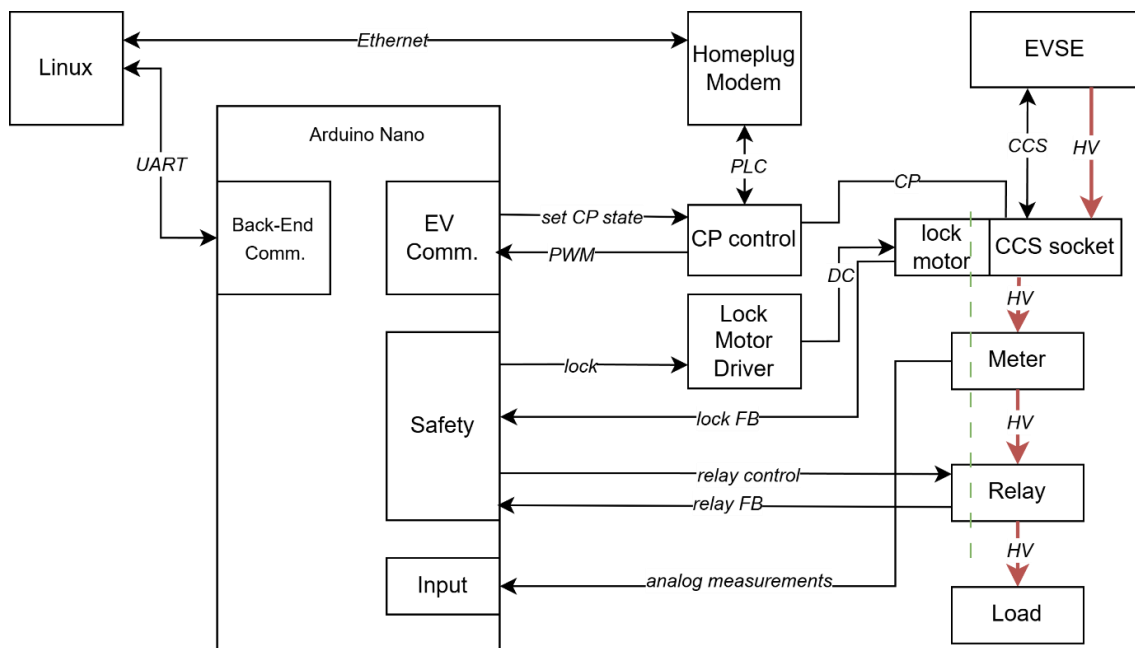


Abbildung 3.1: Hardwarearchitektur, angelehnt an [22]

3.2.2 Software

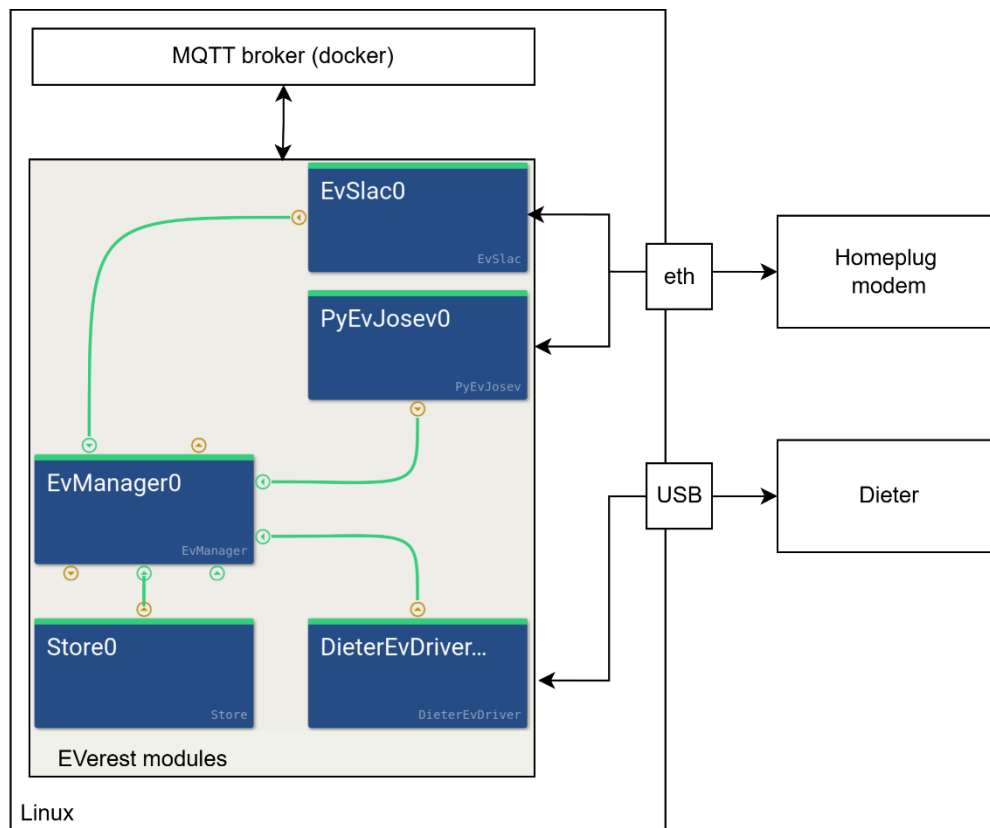


Abbildung 3.2: Softwarearchitektur

Die Software des Systems ist zweigeteilt: zum einen der EVERest stack auf Linux, zum anderen die Low-Level Kommunikation über den Mikrocontroller. Daten werden zwischen den Teilsystemen asynchron per UART ausgetauscht.

Der Mikrocontroller ist dabei rein ausführendes Organ, sämtliche Logik soll in EVERest Module verpackt werden. Motivation hierfür sind maßgeblich zwei Punkte: Zum einen soll das System übersichtlicher sein, indem Logik zentralisiert an einem Ort implementiert wird. Dies soll die Entwicklung und Nachvollziehbarkeit vereinfachen. Zum anderen hat das Praktikabilitätsgründe. Falls eine Änderung vorgenommen wird, müssen nicht EVERest und zusätzlich die „Dieter“ Firmware geändert werden.

EVERest

Die EVERest Modulkonfiguration basiert auf dem EvManager als zentralem Konnektor. Die Fahrzeugsimulation gelingt mit PyEvJosev aus dem EVERest Projekt. Drei der Module haben Zugriff auf Hardware: Das selbstgeschriebene DieterEvDriver greift mittels `ev_board_support` auf einen seriellen Port zu, PyEvJosev und EvSlac arbeiten mit dieselben Ethernet Schnittstelle.

Store ist ein Hilfsmodul und speichert Schlüssel-Wert Paar für EvManager.

3.3 BauteilAuswahl und -auslegung

3.3.1 CCS Stecker

Die von außen offensichtlichste Komponente ist der CCS Stecker. Diese sind naturgemäß sehr groß, um den erforderlichen Strömen standzuhalten. Das ist auch der Grund, warum die Hochvoltkabel direkt im Stecker vercrimpt werden. Einen gebrauchten Stecker mit knapp abgeschnittenen Kabeln wiederzuverwenden ist daher nicht möglich. [23]

Bei einem Neukauf ergeben sich zwei Optionen: das Originalteil von Phoenix Contact für einen hohen dreistelligen Betrag [24] oder einen Klon aus China für etwa ein Viertel des Preises. Hierbei ergibt sich die zusätzliche Herausforderung, dass der Verriegelungsmechanismus nicht mit eingeschlossen ist. Weil die mechanische Kompatibilität zwischen CCS Buchse und Verriegelungsaktuator im Vorhinein nicht offensichtlich ist und wenig Informationen dazu gefunden wurden, wird diese Option nicht weiterverfolgt.

Stattdessen wird das reichliche Angebot an gebrauchten CCS Steckern genutzt. Für weniger als 200 € können einige Bauteile mit Verriegelungsmechanismus und einigen Metern Kabel gefunden werden (z.B. auf ovoko.de). Vorteil hierbei ist, dass es sich um geprüfte Originalteile handelt und man sich eine Crimpverbindung sowie zusätzliches Hochvoltkabel spart. In diesem Projekt wird ein Bauteil aus einem BMW i3 verwendet. Aufgrund des Alters hat dieses einen zulässigen Strom von „nur“ 150 A.

Verriegelung

Während des Ladevorgangs muss das Kabel fixiert werden, um ein Abstecken unter Last zu verhindern. Die Lichtbögen würden zu Schaden am Material führen und auch ein Sicherheitsrisiko darstellen.

Um Ladekabel in der Buchse zu verriegeln, gibt es zwei Ansätze: Per Magnetaktuator oder DC Motor mit Getriebe. In beiden Fällen wird ein Stift in den Stecker geschoben.

Als Notlösung existiert eine Möglichkeit zur manuellen Entriegelung.

Der Magnetaktuator bietet den Vorteil, dass er mechanisch simpler aufgebaut ist und nur eine einfache DC Spannung zum Aktivieren benötigt. Die Rückstellung erfolgt per Feder. Im Gegensatz dazu muss der DC Getriebemotor umgepolst werden, um den Stift wieder in die Ursprungsposition zu bewegen. Die Regelung kann zeit- oder strombasiert erfolgen. [23, S. 3] Die praktische Umsetzung erfolgt über Schaltkreise mit H-Brücke (im Falle des foccci Boards ist es ein DRV8874 [25, S. 4]).

Bei dem gebraucht erworbenen i3 Stecker ist ein Aktuator der Firma Küstner verbaut, bestehend aus einem DC Bürstenmotor, Getriebe und einem Endschalter. Mangels herstellereitigen Alternativen wird eine externe Schaltung auf Basis des (etwas überdimensionierten) L298N Chips verwendet. Während der Verriegelungsmotor typischen Arbeitsstrom von 0,25 A hat, mit Spitzen bis zu 1,5 A, kann der L298N 2 A kontinuierlich arbeiten. [23, S. 3] [26, S. 3]

Die Steuerung erfolgt über 3 digitale Signale: IN1, IN2, ENA

Tabelle 3.1: Bidirektionale Motorsteuerung, nach [26, S. 9]

Eingänge		Funktion
ENA = H	IN1 = H; IN2 = L	Vorwärts
	IN1 = L; IN2 = H	Rückwärts
	IN1 = IN2	Motorbremse
ENA = L	IN1 = X; IN2 = X	Freilauf

Für die Anwendung am CCS Stecker kann ENA permanent mit der Versorgungsspannung verbunden werden.³ Als Feedback dient ein Mikroschalter, den die Micro Controlling Unit (MCU) auslesen kann. [23]

Das verwendete L298N Board beinhaltet ebenfalls einen LDO, der für die Spannungsversorgung der restlichen Platine verwendet werden kann.

3.3.2 CP Messung und Steuerung

Wie in Kapitel 2.1.2 erwähnt, findet die gesamte Kommunikation bei CCS über den CP Kontakt statt. Initial über ein PWM Signal, im zweiten Schritt wird ein hochfrequentes Signal darüber moduliert. Dementsprechend hat der fahrzeugseitige Controller zwei Aufgaben:

1. Messung des PWM Signals (duty cycle)⁴
2. Verändern des CP Spannungspegels

Es sind einige Schutzmaßnahmen nötig, wenn das CP Signal direkt mit einem digitalen Eingang des Prozessors verbunden wird. Zum einen sind die Spannungspegel nicht kompatibel. (5V Logikspannung am MCU ggü. 12 V an CP)⁵, zum anderen darf die Gesamtimpedanz des Fahrzeugs nicht verändert werden. In Abbildung 3.3 ist die Schaltung zur PWM Messung abgebildet. Der CMOS Eingang des Prozessors ist kapazitiv dominiert, weswegen der fließende Strom vernachlässigbar ist. Während die Diode D2 die Signalspannung mit der Versorgungsspannung des MCU begrenzt, regelt R4 in Reihe mit dem Signal den fließenden Strom auf ein Minimum.

³Die Steuerung der Motorgeschwindigkeit erfolgt über PWM an diesem Pin. Das ist aber hier nicht notwendig.

⁴Eine Spannungsmessung ist nicht erforderlich, da EVSE seitig keine Hardware zur Manipulation des CP Spannungsniveaus verbaut ist.

⁵Der negative Anteil wird durch die vorgeschaltete Diode blockiert.

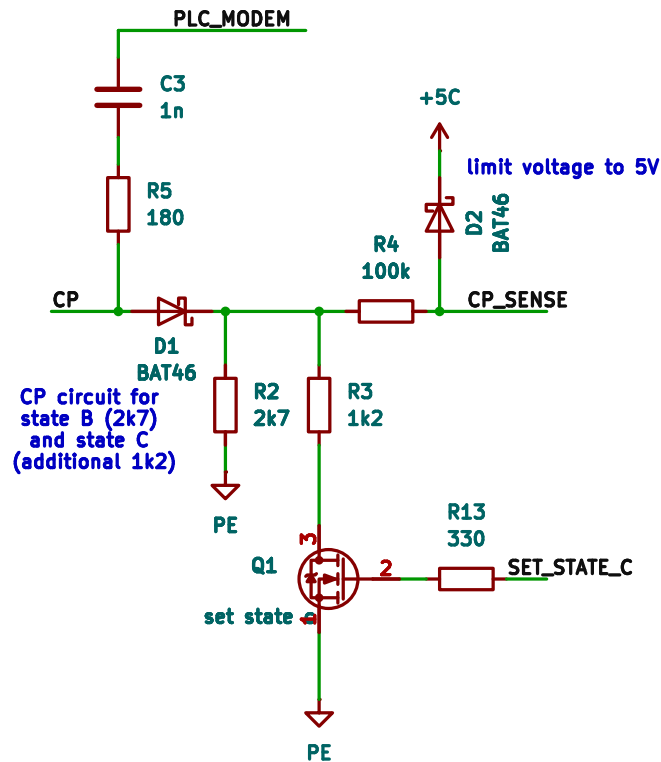


Abbildung 3.3: Schaltplan für CP Messung und Steuerung

Zur Steuerung des CP Spannungspegels genügt es einen weiteren Widerstand R3 zwischen CP und PE zu schalten. Verwendet wird ein einfacher n-Kanal MOSFET als Low-Side Schalter.

Bei deren Umsetzung stellt sich die Frage, ob der PEV Simulator gegenüber der CP Spannung galvanisch isoliert sein muss. Zwar ist das bei anderen CCS Steuergeräten (z.B. foccci, siehe [25]) nicht der Fall, jedoch muss berücksichtigt werden, dass der PEV Simulator nicht potentialfrei im Fahrzeug verbaut ist, sondern netzgebunden mit Energie versorgt wird. Je nach Stromversorgung können Fehlerströme nicht ausgeschlossen werden.

Deshalb wird zunächst der in Abbildung 3.3 gezeigte, einfachere Ansatz verwendet, da der Anstieg an Komplexität wenig Mehrwert bietet.

3.3.3 Leistungsmessung

Isolationsklassen

Bei der Leistungsmessung wird direkt an der Hochspannung gearbeitet, weswegen hier Vorichtsmaßnahmen bezüglich Isolation nötig sind. Für den Automobilbereich gelten dabei die höchsten Anforderungen.⁶ In Datenblättern werden mehrere Begriffe, mit unterschiedlicher Einstufung verwendet, wobei die Folgenden für den PEV Simulator ganz besonders von Relevanz sind:

⁶Kennzeichnung typischerweise durch AEC-Q100

- *Functional insulation* — Insulation needed for the correct operation of the equipment.
- *Basic insulation* — Insulation that provides basic protection against electric shock.
- *Supplementary insulation* — Independent insulation applied in addition to basic insulation in order to ensure protection against electric shock in the event of a failure of the basic insulation.
- *Double insulation* — Insulation comprising both basic and supplementary insulation.
- *Reinforced insulation* — A single insulation system which provides a degree of protection against electric shock equivalent to double insulation.

Im Automobilbereich ist die verstärkte Isolation Standard. Die maximale Spannung, um das oben definierte Isolationsvermögen aufrecht zu erhalten, ist im Betriebsfall:

- *Maximum Rated Working Isolation Voltage* (V_{IOWM}) — An r.m.s or equivalent d.c. voltage assigned by the manufacturer, characterizing the specified long term withstand capability of its isolation.

Diese muss für den PEV Simulator entsprechend der CCS Spezifikation bis zu 1 kV betragen, was die Teilewahl erheblich erschwert. [27]

Strommessung

Die „Dieter“ Hardware soll ebenfalls die Leistung messen können, hierfür gibt es zwei Möglichkeiten: per Shunt oder mit einem Hall-Effekt Sensor. In beiden Fällen muss für galvanische Isolation zum MCU gesorgt werden.

In EVs sind Shunts verbreitet, da mit einem präzisen Widerstand und gut auflösendem ADC selbst bei sehr hohen Leistungen gute Messungen gewonnen werden können. Aus praktischer Sicht ergeben sich die Nachteile, dass zusätzliche Bauteile auf HV Potential verbaut werden müssen und der ADC isoliert sein muss.

Die Variante mit Hall Effekt gestaltet sich dagegen einfacher. Zunächst sollte ein ACS712 verwendet werden, jedoch ist dessen Isolationsvermögen nicht ausreichend. [28, S. 3]

Es gestaltete sich schwierig einen alternativen Hall-Sensor zu finden, nur der TMCS1126 von Texas Instruments erfüllt die Anforderungen für den CCS Spannungsbereich. Nachteil ist, dass das Bauteil für die PCB Montage gedacht ist und somit der Leistungspfad über das PCB führen würde. [29]

Deswegen ist zukünftig geplant, dass ein externer Stromsensor in Form einer nichtinvasiven Klemme auf Basis des Hall-Effekts verbaut wird. Dieser erzeugt eine analoge Spannung, die ganz einfach mit dem internen AD Wandler des Prozessors eingelesen werden kann. Um in der Sensorwahl flexibel zu sein, wird in der „Dieter“ Hardware ein Widerstandsteiler verbaut, um das Spannungsniveau anzupassen, sowie eine Diode D3 vom Eingang Richtung Vcc, die wieder den Eingang vor Überspannung schützt.

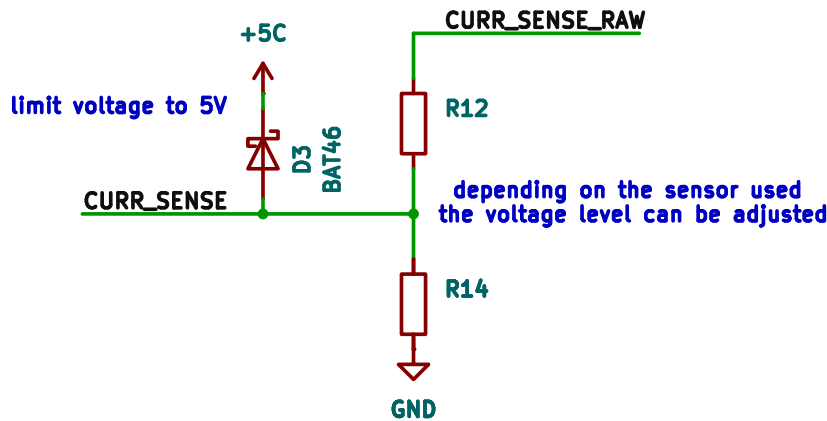


Abbildung 3.4: Schaltplan zum Verarbeiten eines Analogsignals

Spannungsmessung

Bei der Spannungsmessung lässt sich direkter Kontakt mit der HV nicht vermeiden. Uwe Hennig hat das bei pyPLC wie folgt gelöst: Da der m328p Prozessor bereits einen internen AD Wandler hat, wurden für die Low-Level Kommunikation zwei Mikrokontrollern verwendet. Der Analogeingang des einen wurde direkt aus dem HV System gespeist und die Kommunikation über UART mit Optokopplern isoliert. Während die isolierte MCU ausschließlich Daten sendet, empfängt die zweite Einheit Befehle. [11]

Diese Struktur kann in zweierlei Hinsicht optimiert werden. Einen ganzen Prozessor nur als ADC zu verwenden, scheint nicht sehr elegant. Außerdem ist ein relevanter Teil der Platine auf HV Potential. Diesen zu minimieren ist wünschenswert.

Beides kann gelöst werden, indem ein externer ADC mit Isolation der Kommunikation verwendet wird. Als für den Automobilbereich zugelassener ADC wird ein ADS1118Q1 verwendet, mit 16 Bit Auflösung, interner Spannungsreferenz und SPI Anbindung. [30]

Um die SPI Schnittstelle zu isolieren wurde in Betracht gezogen Optokoppler zu verwenden. Da die Schaltfrequenz bei SPI recht hoch ist (im Vgl. zu z.B.: I2C) und zu dem Isolationsvermögen der Optokoppler nicht ausreichende Datenblattangaben existierten, wird sich für eine andere Option entschieden: den dedizierten SPI Isolator ISOW7741Q1. Dieser implementiert vier isolierte Kanäle neben einer integrierten isolierten Stromversorgung, welche für den ADC verwendet werden kann. Somit wird die Anzahl der Bauteile und Oberfläche mit HV Potential bedeutend reduziert. Das Isolationsvermögen ist ebenfalls ausreichend: 1500 VDC sind als Arbeitsspannung angegeben [31]

Die eigentliche Spannungsmessung funktioniert mithilfe eines Spannungsteilers aus Widerständen. Das (isolierte) Erdpotential des ADC wird etwa mittig mit dem Spannungsteiler verbunden, zusätzlich existieren zwei Abgriffe zum Validieren der Messung.

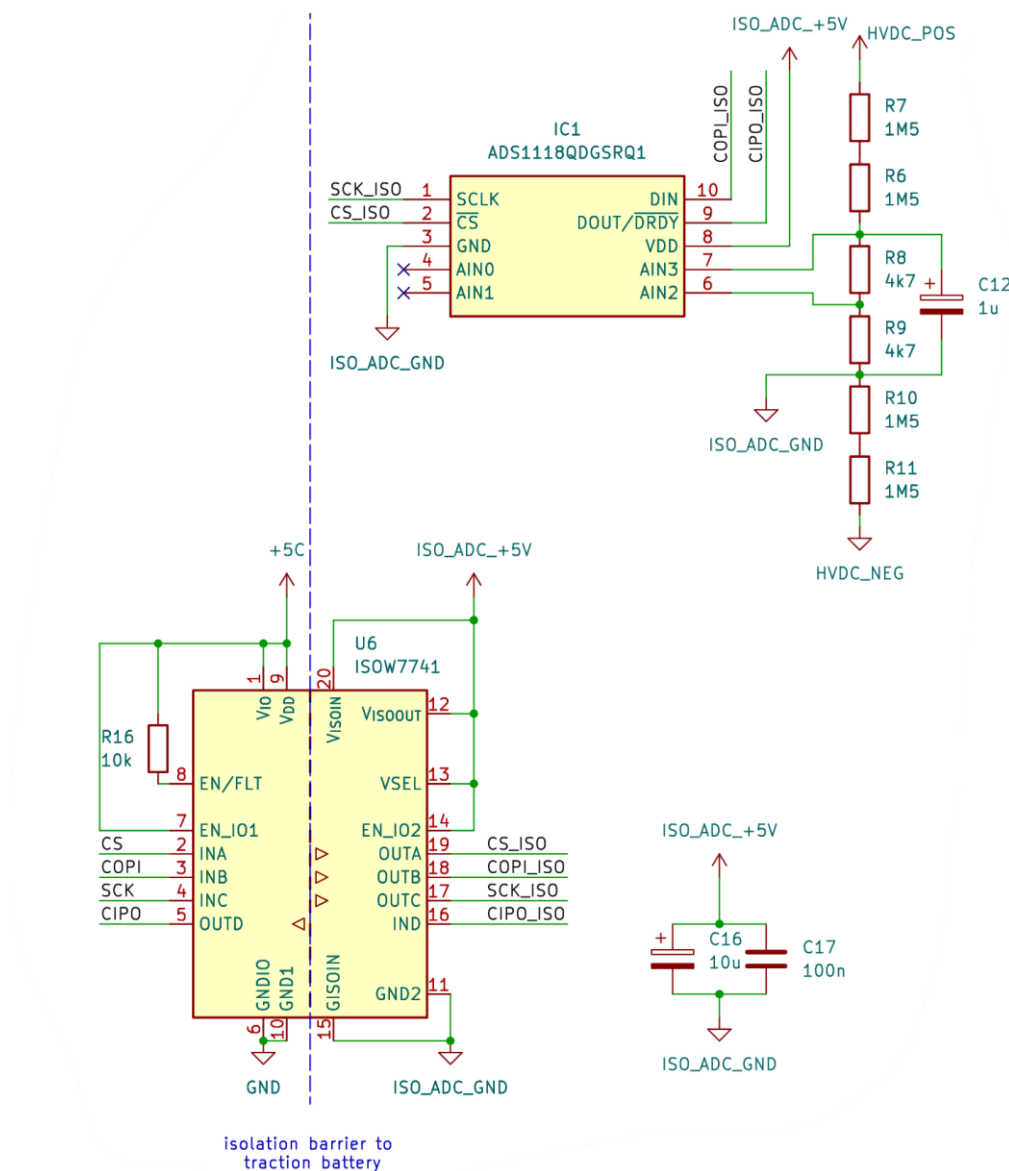


Abbildung 3.5: Schaltplan zur Spannungsmessung

3.3.4 Leistungstrenner und Relais

Für den PEV Simulator werden Bauelemente benötigt, welche zur Not die Last auch unter Nennstrom trennen können. Dafür können Schütze z.B. des Herstellers TDK verwendet werden. Diese haben auch einen kleinen Feedback Schalter, mit dem auf verschweißte Kontakte geprüft werden kann.

Für erste Versuche werden Standard 5V Relais verwendet, diese im nächsten Schritt als Treiber für größere Schütze verwendet werden können. Die „Dieter“ Elektronik hat einen digitalen Eingang, welcher für den Feedback Schalter vorgesehen ist.

Als weiteres Sicherheitselement kann eine Sicherung verbaut werden. Bauteile aus dem PV-Bereich sind wegen der hohen zugelassenen Spannung geeignet, zum Beispiel eine vierpoliger Schutzschalter des Herstellers Protron. [32] Zunächst wird jedoch keine Sicherung verbaut, da im Betrieb eine strombasierte Regelung der EVSE vorliegt und selbst im Falle eines Kurzschlusses kein Schaden entsteht.

3.3.5 SBC und Prozessor

Der PEV Simulator besteht aus zwei Rechnern: ein Linux System als EVERest Basis und ein MCU für die Low-Level Kommunikation.

Für das Linux System soll ein Single Board Computer (SBC) verwendet werden, da die Rechenleistung nicht sehr hoch ist. Hierfür gibt es mehrere Kandidaten, mit dem Raspberry Pi als prominentesten Vertreter.

Ein Ethernet Port für das Homeplug Modem, sowie ein USB Anschluss für die UART Kommunikation mit der „Dieter“ Elektronik haben nahezu alle SBCs. Die RAM Anforderungen von EVERest sind dagegen nicht eindeutig. In der Github Dokumentation werden 512 MB als Mindestanforderung erwähnt [16], an anderer Stelle jedoch auch wesentlich mehr. [33]

Obwohl nicht zwingend erforderlich, soll trotz der höheren Kosten ein Raspberry Pi verwendet werden, für besseren Support und Wiederverwendbarkeit in der Zukunft. Mögliche Alternativen wären z.B. BeagleBone oder Odroid Boards gewesen. Konsequenz davon ist, dass nur die 2 GB RAM Variante gut verfügbar ist. Der benötigte Arbeitsspeicher hängt insbesondere davon ab, ob das EVERest Projekt lokal oder auf einem größeren System gebaut (cross compilation) und dann übertragen werden soll.

Letztendlich wird jedoch kein neuer Pi bestellt, sondern ein bestehender 3B+ verwendet, mit nur 1 GB RAM.

3.3.6 PCB

Im Anschluss werden die oben beschriebenen Bauteile auf ein PCB übertragen, um unnötige Fehlersuche auf einer Lochrasterplatine zu vermeiden und eine schönere Optik zu erreichen. Für zukünftige Funktionen (z.B. für den im CCS vorgesehenen „welded contactor check“) werden entsprechend Anschlüsse eingeplant. Die Verbindung mit externen Komponenten erfolgt mittels Schraubklemmen.

Bauteile, um hardwareseitig Fehler zu generieren, werden jedoch nicht eingebaut. Ein Beispiel wäre ein Widerstand, der einen Leckstrom über Erde simuliert.

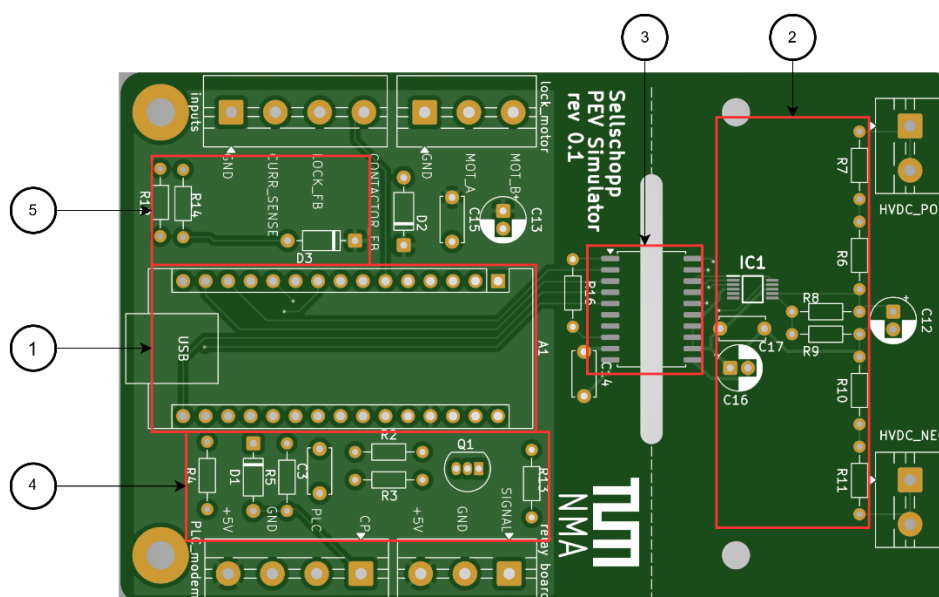


Abbildung 3.6: Visualisierung des PCB

Tabelle 3.2: Funktionsbausteine PCB

Baugruppe	Funktion
1	Microkontroller / Prozessor
2	Hochspannungsmessung
3	Galvanische Isolation zur Hochspannung
4	Regelung des CP Signals
5	Schutzschaltung für Analogeingang

Es ist offensichtlich, dass der Aufbau kein Endprodukt ist. Zwei Umstände führen dazu, dass THT Bauteile verwendet werden. Zum einen war zu Beginn nur eine Lochrasterplatine geplant, zum anderen sind so spätere Modifikationen leichter möglich. Auch der Formfaktor ist recht willkürlich und nicht auf Platz optimiert. So wird kein vollintegriertes Board entworfen, sondern lediglich ein Arduino Board zum Einstecken. Das spart etwas Entwicklungsaufwand und ermöglicht ein einfaches Tauschen des Prozessors, sollte das nötig sein.

Schaltplan und Platinenlayout sind im Anhang B und Anhang C zu finden.

Besonderheiten HV

Durch die hohe Spannung, die im Spiel ist, ergeben sich einige zusätzliche Herausforderungen. Es ist wichtig, folgende Begriffe zu unterscheiden.

Tabelle 3.3: wichtige Begriffe zu Isolation [27]

Abstand	Der kürzeste Abstand zwischen zwei leitenden Teilen, gemessen durch die Luft.
Kriechstrecke	Die kürzeste Entfernung zwischen zwei leitfähigen Teilen, gemessen entlang der Oberfläche einer festen Isolierung. Der kürzeste Weg verläuft in der Regel am Ende des Gehäusekörpers.

Der Mindestabstand ist durch IPC 2221 spezifiziert. Für Spannungen über 500 V gilt folgende Formel, mit 1 kV gleich eingesetzt. [34]

$$1000 \text{ V} - 500 \text{ V} = 500 \text{ V}$$

$$0,25 \text{ mm} + \left(500 \text{ V} \times 0,005 \frac{\text{mm}}{\text{V}} \right) = 2,75 \text{ mm} \quad (3.1)$$

Dies ist jedoch als untere Grenze zu verstehen. Angesichts der Tatsache, dass bei einem Millimeter bereits Lichtbögen durch die Luft entstehen können erscheint das nicht sehr viel.

Die Platine wird quer in einen HV und einen LV Bereich geteilt, einzige Verbindung ist der SPI Isolationschip. Obwohl dieser in der Theorie genug Abstand hat, wird in die Platine ein Schlitz geplant. So besteht selbst bei feuchter Oberfläche oder Verunreinigungen kein Risiko.

Eine weitere Herausforderung ist die Befestigung. Oft werden Schraublöcher verwendet, die mit Erde verbunden sind (das ist auch der Fall in Abbildung 3.6 auf der LV Seite) . An der HV Seite ist dies nicht möglich. Abhilfe bieten Schrauben aus Kunststoff. Da trotzdem davon ausgegangen werden muss, dass Metallschrauben verwendet werden, muss auch ein gewisser Abstand von den Schrauben zu HV Komponenten eingehalten werden.

Der Abstand betrifft auch die häufig verwendeten Kupferflächen, die als Erdung oder Stromversorgung verwendet werden können. Während das um das MCU Sinn macht, stellt das auf HV Seite ein Risiko dar, weshalb sie dort komplett entfernt werden.

Das Standard PCB-Material FR4 ist auch für Hochvoltanwendungen aufgrund des hohen Harzanteils gut geeignet.

3.4 Dieter-Firmware

Aufgabe der „Dieter“ Elektronik ist es die Befehle von EVerest in Hardware umzusetzen, sowie Messwerte zu sammeln. Dies geschieht per UART (was mit dem CH340 Chip über USB emuliert wird).

Es gibt einen Satz an Befehlen (EVerest → MCU) und einen Satz an Messwerten (MCU → EVerest):

Tabelle 3.4: Protokoll MCU - Linux

Befehle		Messwerte	
set_contactor	{0,1}	u_inlet	{0,1,2...}
set_connector_lock	{0,1}	cp_duty_cycle	{0,1,...99,100}
set_state_c	{0,1}	connector_lock_confirmed	{0,1}

Das "Protokoll" ist simpel gehalten, mit Doppelpunkt als Trennzeichen und Zeilenumbruch als Ende ('\n').

Die Referenzimplementierung "YetiEvDriver" von Pionix verwendet ein wesentlich komplexeres Vorgehen. Dort wurden Nachrichten mit protobuf definiert und mithilfe von Parität, COBS [35] und CRC gegen Fehler und mehrdeutiges Auslesen geschützt. [36]

Sobald der Mikrocontroller einen Befehl empfängt, soll dieser umgesetzt werden. Dabei muss beachtet werden, dass das Agieren auf MCU Seite länger dauern kann, der Prozessor deswegen jedoch nicht blockiert werden soll.

Parallel dazu werden in regelmäßigen Abständen alle Messwerte an den Host gesendet.

Um allen Aufgaben gerecht zu werden, wird die Funktionalität in Methoden aufgeteilt, welche regelmäßig aus der Methode `loop()` aufgerufen werden.

```

1. void loop() {
2.   readInletVoltage();
3.   readCpDutyCycle();
4.   readConnLockFB();
5.
6.   serial_rx_task();
7.   connector_lock_task();
8.
9.   if ((millis() - t_last_publish) > PUBLISH_INTERVAL_MS) {
10.    t_last_publish = millis();
11.    publishMeasurements();
12.  }
13. }

```

Die Methode `millis()` gibt die Anzahl an Millisekunden seit Start des Programms zurück und wird verwendet, um regelmäßig die Messwerte per UART zu senden.

`serial_rx_task()` überprüft das serielle Interface und füllt einen Buffer. Sobald ein Zeilenumbruch empfangen wird, wird die Zeile mit validen Befehlen abgeglichen. [Quelle existierender NMA code]

3.4.1 Befehle

Das Steuern des CP MOSEFETs und der Relais ist trivial, während das Verriegeln des Steckers (`lock_connector` Befehl) hingegen etwas komplexer ist: Beim Empfang wird der Motor in die entsprechende Richtung gestartet und das System merkt sich den Zeitpunkt. Die Methode `connector_lock_task()` überprüft regelmäßig den Status und stoppt ggf. den Motor.

```

1. void connector_lock_task() {
2.   if (millis() - lock_timer_start >= LOCK_TIMEOUT_MS) {
3.     // set both inputs equal, this stops motor (see L298 datasheet)
4.     digitalWrite(PIN_MOTOR_DRIVER_A, 0);
5.     digitalWrite(PIN_MOTOR_DRIVER_B, 0);
6.     readConnLockFB();
7.     lock_state = LockState::IDLE;
8.   }
9. }

```

`LOCK_TIMEOUT_MS` wird dem Datenblatt entsprechend auf 600 ms gesetzt.

Tabelle 3.5: relevante Datenblattangaben des Verriegelungsaktuators [23, S. 3]

Locking actuator	
Possible power supply range at the motor	9 V – 16 V
Typical motor current for locking	0,25 A
Reverse current of the motor	Max. 1,5 A
Max. dwell time with reverse current	1 s
Recommended adaption time	600 ms

3.4.2 Messwerte

Zwei der Messwerte, die Spannungs- und Strommessung, erfolgen über einen AD Wandler. Während für die Strommessung der eingebaute 10 Bit Auflösung hat, hat der verwendete externe Wandler 16 Bit und wird über SPI ausgelesen. Im Anschluss erfolgt eine einfache Umrechnung entsprechend der vorgeschalteten Widerstandsteiler.

Die duty cycle Messung gestaltet sich im konkreten Fall recht einfach, da die Frequenz des Signals statisch und bekannt ist. Demzufolge ist die Messung eines einzelnen Spannungspulses, oder anders ausgedrückt die Zeit zwischen der LOW→HIGH und HIGH→LOW Flanke, ausreichend, um auf den duty cycle zu schließen.

Dafür wird die `pulseIn()` Methode aus dem Arduino framework verwendet. Um festzustellen, ob überhaupt ein PWM Signal anliegt, wird ebenfalls die `pulseIn()` Methode mit timeout größer als die Periodendauer verwendet.

In dem Fall, dass tatsächlich PWM anliegt, wird mehrmals gemessen und gemittelt.

```
1. void readCpDutyCycle() {
2.     // three cases to distinguish: logic high, logic low, or 1 kHz PWM
3.     // use pulseIn timeout of 3 ms to rule out PWM
4.     unsigned long t = pulseIn(PIN_CP_DUTY_CYCLE_INPUT, HIGH, 3000);
5.     if (t == 0) {
6.         // constant voltage on cp, as timeout was triggered
7.         // logic high is treated as duty cycle = 100%
8.         if (digitalRead(PIN_CP_DUTY_CYCLE_INPUT)) cp_duty_cycle = 100;
9.         // logic low is treated as duty cycle = 0%
10.        else cp_duty_cycle = 0;
11.    }
12.    else { // else there is a "true" PWM
13.        float tmp = 0;
14.        // averaging PWM
15.        int counter = N_AVERAGE;
16.        for (int i = 0; i < counter; i++) {
17.            int raw = pulseIn(PIN_CP_DUTY_CYCLE_INPUT, HIGH, 2000);
18.            if (raw == 0) counter--; // in case of timeout the reading is omitted
19.            tmp += raw;
20.        }
21.        if (tmp == 0) { // If the sum ist still zero, there is an error
22.            Serial.println("error: invalid cp duty cycle reading");
23.            cp_duty_cycle = -1;
24.            return;
25.        }
26.        // one period is 1 ms => 50 % duty cycle: 500 us => additional division by ten
27.        cp_duty_cycle = (int) tmp / (counter * 10);
28.    }
29. }
```

Bei einem 1 kHz Signal entspräche 50 % duty cycle also einer HIGH Zeit von 0,5 ms.

3.5 EVerest

Die EVerest Software besteht aus separaten logischen Modulen, die einzelne Teile der Funktionalität übernehmen. Der zentrale Konnektor ist `EvManager`, welcher durch folgende Interfaces definiert ist:

Provides:

- `main`: `car_simulator`
- `ev_manager`: `ev_manager`

Requirements:

- `ev_board_support: ev_board_support`
- `ev: ISO15118_ev`
- `slac: ev_slac`
- `powermeter: powermeter`
- `kvs: kvs`

Die Syntax folgt dem Schema „interface_id: interface_name“ Muster und entspricht der EvManager Definition.

Das Interface `car_simulator` implementiert einen Fahrzeugsimulator, vom Ein- bis zum Ausstecken. Der Verlauf der Simulation kann durch einen Wert in der Laufzeitkonfiguration festgelegt werden. Ein Beispiel für einfaches AC-Laden:

```
auto_exec_commands: 'sleep 2;iec_wait_pwr_ready;draw_power_regulated 16,1;sleep 30;unplug'
```

Mit diesen Befehlen wartet das System 2 Sekunden, signalisiert dann der Ladestation Bereitschaft durch Schalten des CP MOSFETs und lässt Stromfluss zu, bevor es nach 30 Sekunden den Vorgang abbricht. Zum jetzigen Stand gibt es hierfür keine dedizierte Liste an Befehlen und Beschreibung, diese müssen aus dem Quellcode gelesen werden.

`ev_manager` hingegen ist nicht direkt relevant, es stellt nur zusätzliche Informationen wie Details über das EV bereit.⁷

Von den benötigten Interfaces ist nur `ev_board_support` zwingend relevant. Dieses leitet Befehle aus dem EVerest System an die Hardware weiter, z.B. um Schütze zu schließen oder das PWM auf dem CP Pin auszulesen. Für den PEV Simulator ist dieses Interface sehr relevant: Maßgeblicher Teil dieser Arbeit war es, ein EVerest Modul zu entwickeln, das an den EvManager „andocken“ kann.

Durch das `ev` Interface kommuniziert EvManager mit dem Modul `PyEvJosev`. Dies ist im Kern eine Python Implementierung eines EVs auf Basis des Josev Projekts. In dessen Konfiguration können u.a. die unterstützten Standards (DIN70121, ISO15118-2 und ISO15118-20) festgelegt werden. `powermeter` und `kvs` können für die Anbindung eines Stromzählers oder eines Schlüssel-Wert Speichers verwendet werden. Das `slac` Interface implementiert das fahrzeugseitige SLAC Protokoll und informiert das System über Veränderungen in der Low-Level Datenübertragung. [37]

Die gesamte Modulstruktur ist in Abbildung 3.2 gezeigt.

3.5.1 Das `ev_board_support` Interface

Das in dieser Arbeit entwickelte Hardware-Support Modul `DieterEvDriver` muss entsprechend der `ev_board_support` Definition einen Satz an Variablen zur Verfügung stellen sowie einen Satz an Befehlen verarbeiten können.

⁷ `ev_manager` bezeichnet das Interface, EvManager das Modul

Variablen

Als Variablen können bei EVerest prinzipiell auch primitive Datentypen verwendet werden, jedoch kommen häufig spezielle Datenstrukturen (data types) zum Einsatz.

Beim `ev_board_support` Interface sind das `bsp_event`, `bsp_measurement` und `ev_info`.

`bsp_event` ist dabei ein enum mit folgenden Einträgen:

```
BspEvent: object
  event: string <required>
  enum:
    - A
    - B
    - E
    - E
    - F
    - PowerOn
    - PowerOff
    - Disconnected
```

Das Treibermodul bestimmt den Systemstatus anhand der Zustände der Schütze und des Verriegelungsmechanismus und "veröffentlicht" diesen per MQTT. EvManager abonniert die entsprechende topic und liest den neuen Wert ein.

Der zweite Datenpunkt `bsp_measurement` besteht aus dem Fehlerstrom in mA, CP duty cycle in Prozent und `proximity_pilot` Objekt. Dieses ist auch ein enum und signalisiert die maximale Stromstärke des Ladekabels:

```
BspMeasurement: object
  Bsp Measurement
  properties:
    proximity_pilot: object <required>
      Current capability (ampacity) of the cable
      properties:
        ampacity: string <required>
          Ampacity value of the cable assembly
        enum:
          - None
          - A_13
          - A_20
          - A_32
          - A_63_3ph_70_1ph
    rcd_current_mA: number <optional>
      RCD current in mA
    cp_pwm_duty_cycle: number <required>
      CP PWM Duty Cycle in percent (0-100)
```

`ev_info` ist weniger von Relevanz und stellt zusätzliche Informationen über das (simulierte) Fahrzeug zur Verfügung.

Befehle

Einige der CMDs werden oder können nicht implementiert werden. Zum Teil sind diese für das Testen mit speziell dafür vorgesehener Hardware gedacht, zum Teil für reine Software Simulation.

Tabelle 3.6: von DieterEvDriver nicht unterstützte Befehle

Befehl	Parameter	Beschreibung
diode_fail	value: <i>boolean</i>	Setting a diode failure
set_ac_max_current	current: <i>number</i>	Setting the mac current requested from the ev
set_three_phases	three_phases: <i>boolean</i>	Setting three phase or one phase support
set_rcd_error	rcd_current_mA: <i>number</i>	RCD current in mA

Die obigen Befehle werden daher nicht weiter betrachtet. Im DieterEvDriver umgesetzt werden:

Tabelle 3.7: von DieterEvDriver unterstützte Befehle

Befehl	Parameter	Beschreibung
enable	value: <i>boolean</i>	Enable/disable the simulation
set_cp_state	cp_state: <i>string</i>	Sets the CP State that should be set by the EV board support driver
allow_power_on	value: <i>boolean</i>	Sets allow_power_on flag. If false, contactor must never be switched on.

allow_power_on ist die letzte Instanz, bevor die Schütze geschlossen werden. Ist dieses nicht auf True dürfen die Schütze niemals an sein, unabhängig von enable und cp_state. set_cp_state bekommt einen String übergeben (A-F) und ist dafür zuständig den CP MOSFET zu aktivieren (Q1 in Abbildung 3.3) [38]

3.5.2 DieterEvDriver Modul

Das Treibermodul implementiert nur das ev_board_support Interface. Die Ordnerstruktur sieht im konkreten Fall so aus: (unter modules/)

```
DieterEvDriver
├── board_support
│   ├── ev_board_supportImpl.cpp
│   └── ev_board_supportImpl.hpp
├── CMakeLists.txt
├── DieterEvDriver.cpp
├── DieterEvDriver.hpp
├── docs
│   └── index.rst
└── manifest.yaml
```

Das Grundskelett stellt EVERest zur Verfügung auf Basis der `manifest.yaml`. Diese beschreibt die Architektur des neuen Moduls:

```
description: EV board support over UART based on uhi22/dieter hardware

config:
  serial_port:
    description: Serial device connected to the Arduino (e.g. /dev/ttyACM0)
    type: string
    default: /dev/ttyACM0
  baud_rate:
    description: UART baud rate
    type: integer
    minimum: 9600
    maximum: 230400
    default: 115200

provides:
  board_support:
    interface: ev_board_support
    description: CP, CCS lock, contactor control via Arduino

enable_telemetry: true

metadata:
  license: https://opensource.org/licenses/Apache-2.0
  authors:
    - Gregor Sellschopp (TUM)
```

Es werden zwei C++ Klassen generiert und mit grundlegenden Methoden gefüllt. `DieterEvDriver.{cpp,hpp}` als Basisklasse und `ev_board_support.{cpp,hpp}` für das Interface. Würde das Modul mehr Interfaces implementieren gäbe es einen Ordner mit Klasse und Header je Interface. Objekte der Basisklasse erhalten eine Referenz auf die Interfaceimplementierung und ein struct mit Konfigurationswerten (festgelegt im `manifest.yaml`). Die Interface Klasse enthält abseits von einer `init()` und `ready()` Methode eine Funktion je Befehl, die Benennung folgt dem Schema `handle_{cmd_name}(...)`. Diese können mit Funktionalität gefüllt werden. Abseits davon können auch weitere Klassen oder Methoden erstellt werden.

Das Ziel bei der Entwicklung war es, dass die gesamte Logik in dem EVERest Modul läuft, und nicht auf dem Mikrocontroller. Wenn eine Änderung notwendig ist, muss dann nicht der Arduino zusätzlich neu programmiert werden.

Deshalb hat die Treiberklasse je CMD-Parameter und VAR eine Zustandsvariable, welche asynchron an EVERest und Arduino geschrieben und gelesen werden.

Für das Empfangen von Daten vom USB wird mit der `boost/asio.hpp` Bibliothek ein separater Thread erstellt, der die Eingaben zeilenweise verarbeitet und bei Verbindungsabbruch einen Neuaufbau versucht. Sobald die Messwerte vom Arduino im Treibermodul empfangen werden, werden diese mittels den von EVERest bereitgestellten Methoden (`publish_bsp_measurement(bspm)`, `publish_bsp_event(bspe)`) auf den entsprechenden MQTT topic geschrieben.

Das Umsetzen der EVERest CMDs auf UART Befehle ist nicht ganz so einfach. Im einfachsten Fall schreibt die `handle_{cmd_name}(...)` Methode direkt den UART Befehl. Eine Schwierigkeit ergibt sich mit dem Befehl `allow_power_on`. Die Schütze sollen betätigt werden je nach Zustand der Steckerverriegelung und des gesetzten CP Zustands. Sobald sich eine der drei Bedingungen ändert prüft die Helfermethode `update_power_state()` alle Zustände und schickt Befehle zum Ein- oder Ausschalten der Schütze.

Die Steckverriegelung wiederum benötigt etwas Zeit, um in Position zu fahren, was von der Methode `verifyLockState(bool lockState)` berücksichtigt wird.

Im Gegensatz zur EVSE Seite besitzt der EvManager kein `connector_lock` Interface, weshalb dessen Logik in das `bsp` Modul eingebaut wird.

3.5.3 Laufzeitkonfiguration

Die Laufzeitkonfiguration ist in `config/config-pev-sim.yaml` gespeichert. Hier werden die Modul- und Interface IDs verknüpft. Im konkreten Fall verweist EvManager beim `ev_board_support` Interface auf das neue Treibermodul.

Der Ablauf des Ladevorgangs wird ebenfalls von EvManager gesteuert. Die Zielspannung bei der Vorladung der Kabel ("precharge") und auch der Nennstrom können hier festgelegt werden.

```
dc_target_current: 5
dc_target_voltage: 200
```

Anhand dieser Parameter wird eine Ladesimulation mit dem Modul `PyEvJosev` gestartet. Der genaue Ablauf entspricht der Befehlskette unter:

```
auto_exec: true
auto_exec_commands: "sleep 1;iso_wait_slac_matched;iso_start_v2g_session DC; iso_wait_pwr_ready;
sleep 36000"
```

Mindestens ein Protokoll kann bei `PyEvJosev` aktiviert werden mit:

```
supported_ISO15118_2: true
```

In dieser Datei werden auch die Konfigurationswerte von allen Modulen gesetzt. Beim dem entwickelten Treibermodul sind das: Ethernet Interface, USB Port und baud rate.

3.6 Build- und Entwicklungswerkzeuge

3.6.1 CMake

Die in C++ entwickelten EVerest Module werden zum Ausführen auf dem Linux System in eine Binärdatei "verwandelt". Hierfür werden Buildgeneratoren verwendet, wovon CMake ein Teil ist.

Im ersten Schritt werden von CMake plattformunabhängige Konfigurationsdateien erstellt. CMake wird daher auch Metabuildsystem genannt. Im zweiten Schritt erzeugt der eigentliche Buildgenerator (z.B. Make) daraus die ausführbare Datei.

CMake basiert auf Skriptdateien (`CMakeLists.txt`), welche an verschiedenen Stellen in der EVerest Dateistruktur zu finden sind.⁸ Diese erfüllen unterschiedliche Aufgaben und sind hierarchisch aufgebaut:

⁸ CMake wird auch beim Installieren von EVerest verwendet. Hierbei werden ausführbare Dateien für die mitgelieferten Module generiert. Typischerweise wird hierfür ein eigener `build/` Ordner verwendet.

- `modules/$MODULNAME/CMakeLists.txt`: gibt an welche `.cpp` Dateien zum Modul gehören
- `config/CMakeLists.txt`: generiert shell Skript für das Projekt
- `./CMakeLists.txt`: Haupteinstiegspunkt; findet Abhängigkeiten (dependencies)

In der Entwicklung wird CMake nicht regelmäßig ausgeführt, sondern nur Make. Wird nur Sourcecode in einem Modul geändert, genügt es den relevanten Teil des Projekts neu zu generieren (sog. partial build):

```
cd $EVEREST_DIR/build
make $SUBDIR -j$(nproc) && make install
```

Dies spart Zeit. Verändert sich dagegen die Projektkonfiguration muss CMake neu ausgeführt werden. Dies ist z.B. der Fall bei

- Änderung von `CMakeLists.txt`
- Neue / andere Abhängigkeiten (`dependencies.yaml`)
- neue Module (`ev_add_module()`)
- Änderung von CMake Optionen beim build (flags)

cross-compilation

Wenn in der Entwicklung C++ Projekte mit CMake gebaut werden wird aus sourcecode eine ausführbare Datei erstellt. Die Prozessorarchitektur entspricht implizit der des Hosts, ist aber auch konfigurierbar. Das ermöglicht zum Beispiel, dass auf einem stärkeren Entwicklungsrechner ein Image für ein eingebettetes System entwickelt wird.⁹ Projekte direkt auf einem eingebetteten System zu bauen ist unüblich und technisch oft auch nicht möglich, da Ressourcen sehr begrenzt sind. Der in diesem Projekt verwendete Pi hat nur 1 GB RAM.

Dabei wird der modulare Charakter des Generierungsprozesses genutzt. Mit einer weiteren Konfigurationsdatei werden z.B. Details des Zielsystems angegeben. Für den Raspberry Pi als Zielsystem können online toolchains gefunden werden. Mithilfe derer können Programme für eine andere Prozessorarchitektur gebaut werden.

Das cross-kompilieren in dieser Weise hat mit einfachen C++ Projekten funktioniert, mit dem gesamten EVerest Projekt inklusive DieterEvDriver nicht. [39] [40] [41], [42], [43]

3.6.2 Yocto

Das Yocto Projekt ist der empfohlene Weg, um EVerest auf einem eingebetteten Systems zuinstallieren. Yocto bezeichnet ein framework aus verschiedenen Werkzeugen, Metadaten und Prozessen, um wiederholbar maßgeschneiderte Linux Distributionen zu generieren. Bitbake fungiert als build Generator (in manchen Aspekten ähnlich zu Make) und arbeitet mit in "Schichten" strukturierten Rezepten (`*.bb`). Als Ergebnis können viele unterschiedliche Formate

⁹ Oft host: x86, target: ARM

vorkommen (tarball, Linux Pakete, Bootloader). Für diesen Anwendungsfall ist ein *.wic.bz2 geeignet, welches nach dem Entpacken direkt auf eine SD Karte kopiert werden kann.

Für das PEV Simulator Projekt ist die Idee, mit Yocto ein Image für den Raspberry Pi 3 zu generieren, das nicht nur EVerest selbst, sondern auch das selbst geschriebene Modul DieterEvDriver sowie hilfreiche Debugging Werkzeuge enthält. Abseits davon soll es vorkonfiguriert werden, um sich wiederholende Schritte beim ersten Start zu vermeiden und ein reproduzierbares Ergebnis zu erhalten

Ein weiterer großer Vorteil: sollte das System nicht mehr ordnungsgemäß funktionieren, kann das existierende Image neu installiert, und direkt weitergearbeitet werden.

Die Umsetzung erfolgt entlang des EVerest Leitfadens, wobei viele layer entsprechend benötigter Funktion hinzugefügt werden. Diese werden festgelegt in conf/bblayers.conf

```

1. # POKY_BBLAYERS_CONF_VERSION is increased each time build/conf/bblayers.conf
2. # changes incompatibly
3. POKY_BBLAYERS_CONF_VERSION = "2"
4.
5. BBPATH = "${TOPDIR}"
6. BBFILES ?= ""
7.
8. BBLAYERS ?= " \
9.   /home/$USER/yocto-everest/poky/meta \
10.  /home/$USER/yocto-everest/poky/meta-poky \
11.  /home/$USER/yocto-everest/poky/meta-yocto-bsp \
12.  /home/$USER/yocto-everest/poky/meta-openembedded/meta-oe \
13.  /home/$USER/yocto-everest/poky/meta-openembedded/meta-python \
14.  /home/$USER/yocto-everest/poky/meta-openembedded/meta-networking \
15.  /home/$USER/yocto-everest/poky/meta-raspberrypi \
16.  /home/$USER/yocto-everest/poky/meta-everest \
17.  /home/$USER/yocto-everest/poky/meta-pev_sim \
18.  /home/$USER/yocto-everest/poky/meta-openjdk-temurin \
19.  "

```

meta-raspberrypi wird als BSP eingefügt, Java und Python werden für den Fahrzeugsimulator benötigt.

meta-pev-sim ist eine selbst geschriebene BitBake layer und abstrahiert Funktionen aus der Hauptkonfiguration in eine eigene Schicht. Somit wird sichergestellt, dass DieterEvDriver referenziert, die richtigen Treiber installiert und z.B. die WLAN Schnittstelle korrekt konfiguriert ist.¹⁰

Die zweite wichtige Konfigurationsdatei ist conf/local.conf. Einige wichtige Veränderungen gegenüber der EVerest Vorlage sind:

- Anstelle von EVEREST_RELEASE_PACKAGES wird IMAGE_INSTALL:append verwendet um zusätzliche Linux Pakete miteinzubinden (z.B. mosquito oder Python)
- MACHINE = "raspberrypi3-64" setzt die richtige Prozessorarchitektur. Diese sind in meta-raspberrypi definiert
- EXTRA_IMAGE_FEATURES += "debug-tweaks" ermöglicht u.a. das Anmelden ohne Passwort, was in der Entwicklung praktisch ist

¹⁰ https://github.com/gregsell/DieterEvDriver/tree/main/yocto/scarthgap/meta-pev_sim

Die Datei ist vollständig im Anhang D zu finden. [45], [46], [47]

In der fertigen Installation wird EVerest systemweit unter `/bin/manager` installiert und kann - im Gegensatz zur Entwicklungsmaschine – von überall aufgerufen werden. Die mitgelieferten Laufzeitkonfigurationen sind unter `/etc/everest`, während die vorkompilierten Modulbinaries unter `/usr/libexec/everest` abgelegt werden.

Falls im Nachhinein Module manuell für ARM kompiliert werden können diese hier eingehängt werden.

Mit Yocto bzw. BitBake Betriebssystem-Images zu generieren ist aufwendig. Ein möglichst starker Rechner und genug Festplattenspeicher sollten zur Verfügung stehen. Nach dem build von `core-image-full-cmdline`, das grundlegende CLI Tools enthält, sind beinahe 100 GB Speicher belegt.¹¹ Ein Großteil davon sind Buildartefakte, die final nicht gebraucht werden, jedoch für das Debuggen nützlich sind und beim folgenden build wiederverwendet werden können.

3.6.3 Open-plc-utils

Das Homeplug Modem vom Typ TPlink TL-PA4010P v5.0 wird als existierende Computing Einheit übernommen. Herausforderung hierbei ist, dass die werksseitige Firmware die SLAC Nachrichten nicht zum Ethernet Port durchreicht. Dafür muss ein patch angewendet werden.

Hierfür geeignet ist das `open-plc-utils` Toolkit. Dies ist eine Ansammlung von kleinen Programmen, die jeweils bestimmte Funktionen erfüllen, um Konfiguration oder Firmware zu verändern. Die Struktur ist sehr kompakt und ist gedacht für die Anwendung in Skripten. Sowohl die Programme, als auch die zugehörigen Bedienungsanleitungen werden lokal aus dem Quellcode erstellt.

Das oben genannte TP Link Modell verwendet das AR7420 chipset, dafür ist `plctool` das richtige Programm.

Open-plc-uis verwendet zwei proprietäre Dateiformate:

- *.pib : parameter information block : Konfigurationswerte
- *.nvm : non-volatile memory : eigentliche Firmware

Um das Modem SLAC fähig zu machen, muss die .pib Datei geändert werden. Dazu wird zunächst die aktuelle Datei ausgelesen und bestimmte Felder überschrieben. [9], [11]

```
# Auslesen der Konfiguration
$ sudo plctool -i <eth interface> -p factory.pib local

# Datei duplizieren und modifizieren:
$ cp factory.pib pev.pib
$ setpib pev.pib 74 hfid "PEV"
$ setpib pev.pib F4 byte 1
$ setpib pev.pib 1653 byte 1
$ setpib pev.pib 1C98 long 10240 long 102400

# schreiben der neuen Datei
$ sudo plctool -i <eth interface> -P pev.pib local
```

¹¹ <https://everest.github.io/latest/explanation/linux-yocto/setting-up-linux-os.html>

3.6.4 Venv & ev-cli

Standard bei Linuxprojekten ist es packages nicht systemweit zu installieren, sondern je Projekt in einer virtuellen Python Umgebung. ("virtual environment" - venv). So werden auch die EVerest Abhängigkeiten und einige benötigte Werkzeuge in einem venv installiert. In der Praxis muss das venv aktiviert werden mit:

```
$EVEREST_DIR/build:~$ source venv/bin/activate
```

Nach diesem Schritt stehen einige Werkzeuge zur Verfügung, so auch ev-cli. Diese kann das Codeskelett für zu entwickelnde Module oder Interfaces autogenerieren. Sowohl die benötigten Header und Klassendateien als auch die Methodenstruktur innerhalb werden von ev-cli erstellt. In den Dateien werden Markierungen platziert, wo benutzerdefinierter Code zwischen den ev-cli Aufrufen erhalten bleibt.

Sobald die yaml Konfiguration von Interfaces und Modulen abgeschlossen ist, werden mit diesem Hilfsprogramm die noch leeren Klassen und Header Dateien erzeugt:¹²

```
├── modules
│   └── Modul_A
│       ├── CMakeLists.txt
│       ├── Modul_A.cpp
│       ├── Modul_A.hpp
│       ├── doc.rst
│       ├── docs
│       │   └── index.rst
│       ├── interface_X
│       │   ├── interface_XImpl.cpp
│       │   └── interface_XImpl.hpp
```

Für jedes Modul wird eine Klasse mit Namen `${MODULNAME}. {cpp, hpp}` generiert. Diese enthält abseits vom Konstruktor einige Methoden zur Initialisierung und Konfiguration durch EVerest, ist sonst aber frei erweiterbar. [18]

Jedes implementierte Interface erhält ebenfalls eine Klasse mit einem Skelett an Methoden, die der Definition des implementierten Interfaces entsprechen (generierte Name: `${MODUL_NAME}/${INTERFACE_NAME}Impl. {cpp, hpp}`). Aufgabe des Entwicklers ist es nun, diese Methoden mit Funktionalität zu füllen. Durch die automatische Generierung des Codeskeletts und die standardisierte Beschreibung der Schnittstellen werden Fehler minimiert und Kompatibilität gewährleistet.

In der EVerest Dokumentation wird bei jedem neuen Projekt die EVerest source zusätzlich in das Projektverzeichnis geklont. Dies kann vermieden werden, indem bei ev-cli die Option `--everest-dir ~/path/to/everest-core` verwendet wird. [48]

¹² Standardmäßig C++, es sind aber auch andere Sprachen möglich

3.6.5 Admin panel

Ein weiteres nützliches Tool aus dem EVerest framework ist das admin panel. Dies dient zur einfacheren Erstellung von Laufzeitkonfigurationen. Während diese auf yaml basieren, gibt es das admin panel um mit graphischer Oberfläche zu bearbeiten. Beispielsweise können neue Module hinzugefügt und über die geeigneten Schnittstellen verbunden werden. Auch das Setzen der Konfigurationswerte einzelner Module ist einfach möglich.

Es gibt zwei Möglichkeiten das admin panel zu benutzen. Die neueste Version ist stets auf github pages veröffentlicht und ist für schnelle Tests sehr praktisch. Außerdem startet eine lokale admin panel Instanz auf port 8849 sobald EVerest gestartet wird. Typischerweise findet sich diese Information auch in den EVerest Logs.

Die in Abbildung 3.2 enthaltene Modulübersicht entstammt ebenfalls dem admin panel. [49], [50]

3.6.6 MQTT Explorer

Die gesamte EVerest Kommunikation läuft über einen MQTT broker (mosquitto). MQTT Explorer ist ein open source Werkzeug zur graphischen Aufbereitung der MQTT Kommunikation. Nach dem Verbinden mit dem broker werden alle Nachrichten nach topic sortiert und können über die Zeit nachverfolgt werden.

Auch aktives Eingreifen ist über MQTT Explorer möglich. Es kann manuell eine topic angegeben werden und eine Nachricht mit beliebigem Anhang geschickt werden. Ganz konkret: Möchte man manuell den Zustand der Hardware mittels ev_board_support Interface setzen (z.B. cp_state: B), kann folgende Nachricht aus MQTT Explorer veröffentlicht werden:

```
topic:
everest/modules/DieterEvDriver_instance/impl/board_support/cmd/set_state_c

payload:
{
  "data": {
    "args": {
      "cp_state": "B"
    },
    "id": "372c0123-9b68-45e1-9255-8083ae4c0224",
    "origin": "manual test"
  },
  "msg_type": "Cmd"
}
```

Dies hat den Vorteil, dass schnell getestet werden kann, ohne vorher jedes Detail implementieren zu müssen.

3.6.7 Wireshark

Wireshark ist eine freie Software zum Mitschnitt und graphischen Aufbereitung von Datenprotokollen und hat sich zum Standard der Netzwerkanalyse entwickelt. Es unterstützt verschiedenste Schnittstellen, u.a. das lokale Netzwerk, Bluetooth und USB.

Mit einem geeigneten Plugin kann die gesamte HLC zwischen Fahrzeug und Ladesäule dekodiert werden. Nach Farbe aufgeschlüsselt lassen sich leicht die einzelnen Phasen erkennen. [51], [52]

Eine Herausforderung beim PEV Simulator Projekt, bei der sich Wireshark als besonders nützlich erwies, war die Fehlersuche bei der initialen Verbindung zwischen PLC Modem und Linux. Sobald eine physikalische Verbindung hergestellt wird, versucht der Linux Host per DHCP dem

Ethernet Adapter eine IP Adresse zuzuweisen. Normalerweise ist dies das gewünschte Verhalten und geht mit einem schnellen Verbindungsaufbau einher. Im konkreten Fall ist dies jedoch nicht der Fall, da die Schnittstelle aktiv von der Software verwaltet wird. Das Einmischen des NetworkManagers führt zu sehr langer Wartezeit und meist Verbindungsabbruch. Auf Linux lässt sich dann Folgendes beobachten:

```
gregor-sellschopp@GS:~/everest/NMA_PEV_SIM$ nmcli d
DEVICE          TYPE      STATE              CONNECTION
wlp4s0          wifi      verbunden          NMA_EVSE
enx000acd2d04f2 ethernet  wird verbunden (IP-Einstellungen werden ermittelt) Kabelgebundene Verbindung 2
br-0255813afcc4 bridge    verbunden (extern) br-0255813afcc4
br-c609e3492545 bridge    verbunden (extern) br-c609e3492545
lo              loopback  verbunden (extern) lo
docker0        bridge    verbunden (extern) docker0
p2p-dev-wlp4s0  wifi-p2p  nicht verbunden    --
enp0s31f6       ethernet  nicht verfügbar    --
cdc-wdm0        gsm       nicht verfügbar    --
tailscale0     tun       nicht verwaltet     --
```

Abbildung 3.7: Fehlerhafte IP Zuweisung des Homeplug Modems

Diese Erkenntnis ging aus Wireshark Mitschnitten hervor. Die einfache Lösung war, das Interface von NetworkManager mittels einer Konfigurationsdatei ignorieren zu lassen.

```
# /etc/NetworkManager/conf.d/homeplug-unmanaged.conf
[keyfile]
unmanaged-devices=interface-name:<iface>
```

Die MAC Adresse kann mit `ip link show` ermittelt werden. Danach muss der NetworkManager nur noch neu gestartet und das Interface aktiviert werden.¹³

```
sudo systemctl restart NetworkManager
ip link set <iface> up
```

3.6.8 PlatformIO

Um einen Mikrocontroller zu programmieren gibt es viele Möglichkeiten, vermutlich die einfachste davon die Arduino IDE. PlatformIO funktioniert ähnlich, ist jedoch bedeutend flexibler. Im Kern sind es gebündelte Werkzeuge zur Entwicklung auf eingebetteten Systemen, welche eine relativ unabhängige Hardwareentwicklung ermöglichen. Die Konfiguration erfolgt (ggü. der Arduino IDE) über eine .ini yaml Datei.

Für die Programmierung des Arduinos in der „Dieter“ Elektronik wird ebenfalls PlatformIO verwendet. Als Plugin für VSCodium ist es sehr einfach in den Workflow miteinzubinden. [53], [54]

¹³ Der letzte Schritt ist nach jedem Neustart der Verbindung notwendig. Ein entsprechendes cron Skript bietet sich an.

4 Ergebnisse

Bei der Inbetriebnahme der Hardware werden zunächst alle Einzelfunktionen der Hardware getestet. Beim Testen an einer einfachen AC Ladesäule wird die PLC nicht benötigt und die Funktion des CP MOSFETs, und der PWM Messung via `pulseIn` (siehe Absatz 3.4.2) kann verifiziert werden. Da bei einer DC Ladesäule stets 5 % PWM vorliegen, eignet sich eine AC Station, bei der die Ladeleistung verändert und die Messwerte beobachtet werden können.

Die Spannungsmessung via externem ADC und Isolationschip funktioniert nicht ohne Weiteres. Aufgrund fehlender Leiterbahnen auf der Platine hat der interne DCDC Wandler des Isolationschips ISOW7441 eine falsche Spannungsreferenz, weshalb dieser 8 V anstatt 5 V erzeugt und überhitzt. Diese hohe Spannung zerstörte ebenfalls den AD Wandler. Die Schaltungspläne werden korrigiert, eine zweite Iteration wird jedoch nicht gefertigt.

EVERest benötigt die gemessene Spannungswerte nicht, weshalb das Ausführen einer Ladesimulation trotzdem möglich ist.

Der nächste Schritt ist das DC Laden. Hier ist das Aushandeln der IP Adressen (SLAC matching) ein wesentlicher Bestandteil des Verbindungsaufbaus, bevor am Combo-2 Stecker Spannung messbar ist. Nachdem das Problem mit dem NetworkManager gelöst und das Homeplug Modem gepatcht wurde (siehe Kapitel 3.6.7 und 3.6.3), war dies wiederholbar möglich. Parameter für die Ladesimulation können in der Laufzeitkonfiguration (siehe Kapitel 3.5.3) erfolgreich festgelegt werden.

Der Gesamtaufbau wird in einen Schaltschrank gebaut, inklusive einem CCS Stecker. Die Befestigung erfolgt mittels Hutschiene. Für den SBC, Dieter, Homeplug Modem und Motortreiber werden Halterungen 3D-gedruckt.¹⁴

¹⁴ https://github.com/gregsell/Dieter_PEV_Simulator/tree/main/docs

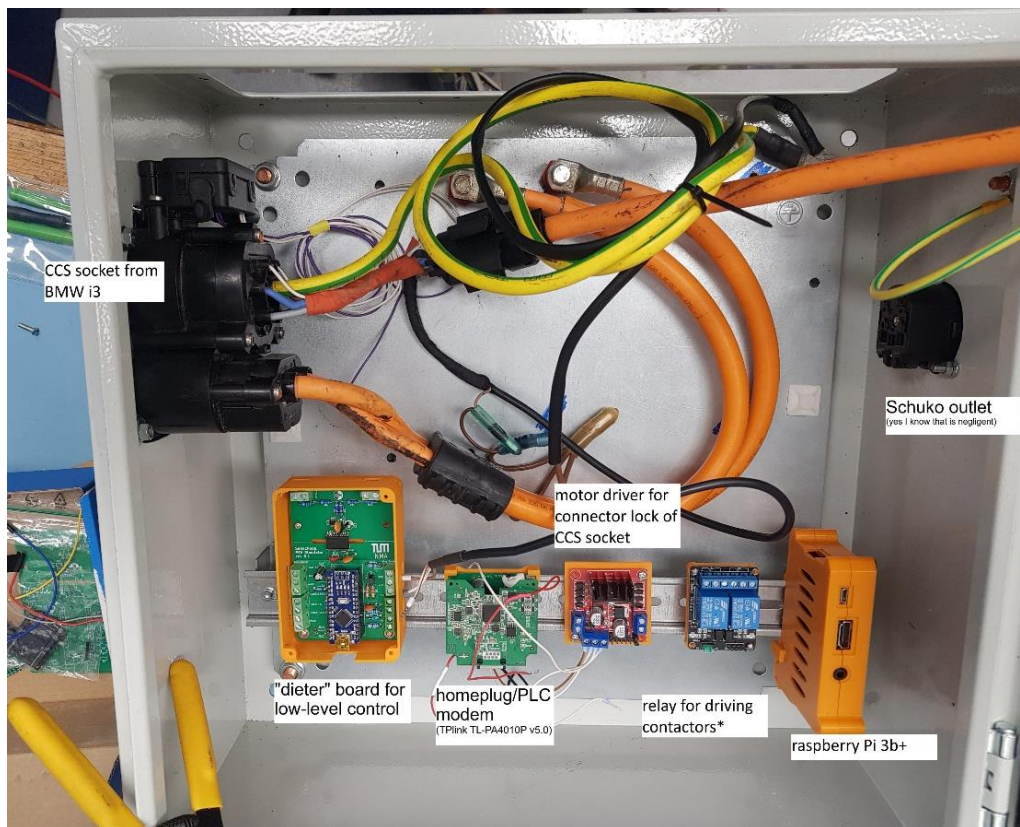


Abbildung 4.1 Bau des Fahrzeugsimulators

Das in dieser Arbeit entwickelte Modul DieterEvDriver, welches das `ev_board_support` Interface bereitstellt, erfüllt seine Aufgabe im Betrieb ohne Probleme.

In bestimmten Randsituationen ist noch zusätzliche Logik denkbar. Wenn die Ladesimulation außerplanmäßig beendet wird, weil der EVERest Prozess abbricht, ist das Verhalten der Schütze und Verriegelung noch nicht konsistent. Dies ist nicht nur relevant bei einem Crash, sondern auch wenn der Prozess manuell im Terminal beendet wird.

Der Sourcecode von „Dieter“ Firmware und EVERest Treibermodul sind auf GitHub in jeweils eigenen repositories.¹⁵ Logs von DC Ladesimulationen an sowohl EVERest basierten Ladesäulen als auch öffentlichen Alpitronic Geräten sind auf Github unter `gregsell/DieterEvDriver/logs` zu finden. Dort sind jeweils erfolgreiche und gescheiterte Simulationen zu finden, um für unterschiedliche Fehlerbilder eine Referenz zu haben.

¹⁵ https://github.com/gregsell/Dieter_PEV_Simulator/, <https://github.com/gregsell/DieterEvDriver>



Abbildung 4.2: Tee kochen am Alpitronic HYC-300

Damit das EVERest Projekt auf dem zur Verfügung stehenden Raspberry Pi laufen kann, wird eine neue BitBake layer geschrieben und eine maßgeschneiderte Linux Distribution erzeugt. Das fertige Image ist voll funktionsfähig und kann EVERest Projekte auf Hardware mit nur 1 GB RAM ausführen. Jegliche zusätzlich referenzierten Pakete (u.a. Java, Mosquitto, tshark) funktionieren in Verbindung mit EVERest.

Bitbake Konfiguration und Image sind im DieterEvDriver repository unter `/docs/yocto` zu finden. Auch das fertig kompilierte Image ist hochgeladen und unter „releases“ verfügbar.

Die Auseinandersetzung mit Yocto erfolgte, da keine passende Hardware zur Verfügung stand. Von einem Seitenprojekt, das nur „Mittel zum Zweck“ sein sollte wurde der Aufbau sehr schnell kompliziert. Solch ein vollintegrierter Yocto workflow ist vor allem für die Produktion gedacht und bei einem einmaligen Aufbau nicht lohnend, für einen gut dokumentierten Forschungsaufbau jedoch der richtige Weg.

5 Zusammenfassung, Schlussfolgerungen und Ausblick

In dieser Arbeit werden zunächst Grundlagen des CCS Protokolls und der verwendeten Stecker erläutert. Unterschieden wird die Analoge Kommunikation mit dem PWM Signal auf dem CP Pin sowie die PLC, die für das DC Laden erforderlich ist. Zwei bestehende CCS Implementierungen werden analysiert und hinsichtlich Flexibilität verglichen.

Im Anschluss werden Anforderungen an den PEV Simulator gestellt und die Systemarchitektur dargelegt. Der Aufbau kann in zwei Unterprojekte geteilt werden. Zum einen das Linux System, auf dem EVerest läuft, zum anderen der Mikrocontroller samt Firmware, der den Aufbau physikalisch mit dem CCS verbindet.

Der Schaltplan der Interfaceelektronik „Dieter“ wird in KiCAD entworfen, während für die Entwicklung der Firmware Platformio verwendet wird.

Im Kontext von EVerest werden in dieser Arbeit das Hardwaresupportmodul „DieterEvDriver“ sowie funktionale Laufzeitkonfigurationen für AC und DC Laden entwickelt. Im zweiten Schritt wird mithilfe des Yocto Projekts erfolgreich ein Linux Image für den Raspberry Pi 3b+ generiert und auch hierauf die EVerest Fahrzeugsimulation ausgeführt.

EVerest als Ökosystem ist für die Entwicklung eines Fahrzeugsimulators geeignet und grundsätzlich sehr flexibel. Mit der Flexibilität kommt wie so oft auch die Komplexität. Vor allem der Einstieg ist komplex aufgrund des nicht sehr breiten Supports und nicht immer konsistenten Dokumentation. Die Auseinandersetzung mit Yocto ist lehrreich, zeitaufwendig, aber der richtige Weg.

An einigen Punkten kann der Aufbau hinsichtlich Sicherheit und Funktionalität noch verbessert werden. So bleiben zum Beispiel die Schütze geschlossen, wenn der EVerest Prozess stirbt. Dies könnte gelöst werden durch ein regelmäßiges Signal vom Treibermodul zum MCU („heartbeat“). Sobald der Prozess abstürzt, kann die MCU einen sicheren Betriebspunkt herstellen. Zusätzlich kann noch ein Leistungstrenner nebst Hochleistungsschützen verbaut werden, um höhere Ströme zu schalten. Auch das Steckersystem am Ausgang entspricht nicht den höchsten Standards und kann verbessert werden.

Schlussendlich funktionierte dieser Aufbau sehr zuverlässig, sowohl an der EVerest basierten Ladesäule als auch in Verbindung mit Geräten von Alpitronic.

6 Literatur

- [1] Wikipedia. "Combined Charging System." Zugriff am: 24. April 2026. [Online.] Verfügbar: https://de.wikipedia.org/w/index.php?title=Combined_Charging_System&oldid=266378734
- [2] evchargerpile. "Foto Typ 1 Combo CCS Stecker." Zugriff am: 25. April 2026. [Online.] Verfügbar: <https://www.evchargerpile.com/de/video/products-detail-778739>
- [3] gcharin, "design_guide_combined_charging_system_v7," S. 1–115, 2019. [Online.] Verfügbar unter: https://www.charin.global/media/pages/technology/ccs-specification/c07034e41e-1626949173/design_guide_combined_charging_system_v7.pdf
- [4] chademo. "JAE | V2H KW02C | CHAdeMO." Zugriff am: 25. April 2026. [Online.] Verfügbar: <https://www.chademo.com/products/connectors/e-v2h>
- [5] Uwe Hennig, Johannes Hübner, Linux Info Tag Augsburg. "Open Source CCS Implementierung zur Nutzung der Schnellladeinfrastruktur." Zugriff am: 25. April 2026. [Online.] Verfügbar: <https://www.youtube.com/watch?v=Bc7IXPGtLXY>
- [6] BNA. "Bundesnetzagentur - Ladesäulenkarte." Zugriff am: 25. April 2026. [Online.] Verfügbar: <https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/E-Mobilitaet/Ladesaeulenkarte/start.html>
- [7] Wikipedia. "GB/T charging standard." Zugriff am: 25. April 2026. [Online.] Verfügbar: https://en.wikipedia.org/w/index.php?title=GB/T_charging_standard&oldid=1310785854
- [8] C. e. V. "CharIN Stands Behind CCS and MCS, but also supports the standardization of Tesla NACS – CharIN." Zugriff am: 25. April 2026. [Online.] Verfügbar: <https://www.charin.global/news/charin-stands-behind-ccs-and-mcs-but-also-supports-the-standardization-of-tesla-nacs/>
- [9] GitHub: Qualcomm Atheros. "qca/open-plc-utils: Qualcomm Atheros Open Powerline Toolkit." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://github.com/qca/open-plc-utils>
- [10] openinverter Forum. "Open source CCS using AR7420 - openinverter forum." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://openinverter.org/forum/viewtopic.php?p=37085#p37085>
- [11] GitHub. "uhi22/pyPLC: Hardware for CCS Sniffing." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://github.com/uhi22/pyPLC/blob/master/doc/hardware.md>

- [12] Github: uhi22. "pyPLC/doc at master · uhi22/pyPLC." Zugriff am: 24. April 2026. [Online.] Verfügbar: <https://github.com/uhi22/pyPLC/tree/master>
- [13] Github: uhi22. "uhi22/dieter: Device Interface ElecTronic Especially not limited to Raspberry." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://github.com/uhi22/dieter/blob/master/dieter.ino>
- [14] Github: uhi22. "uhi22/foccci: Fully Open CCS Charge Controller Interface with QCA7005 PLC modem." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://github.com/uhi22/foccci>
- [15] openinverter shop. "openinverter shop: foccci." Zugriff am: 26. April 2026. [Online.] Verfügbar: https://openinverter.org/shop/index.php?route=product/product&product_id=79
- [16] everest.github.io. "High-Level Overview of EVerest - EVerest Manual 2026.02.0." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/explanation/high-level-overview.html>
- [17] mintlify: EVerest Core. "Interface System - EVerest Core." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://mintlify.wiki/EVerest/everest-core/core-concepts/interfaces>
- [18] everest.github.io. "EVerest Modules in Detail - EVerest Manual 2026.02.0." Zugriff am: 26. April 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/explanation/detail-module-concept.html>
- [19] mintlify: everest modules, "Modules Overview," *EVerest Core*. Zugriff am: 27. April 2026. [Online.] Verfügbar: <https://mintlify.wiki/EVerest/everest-core/modules/overview>
- [20] everest.github.io. "PhyVersoBSP - EVerest Manual 2026.02.0." Zugriff am: 27. April 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/reference/modules/HardwareDrivers/EVSE/PhyVersoBSP/autogenerated.html>
- [21] everest.github.io. "EvManager - EVerest Manual 2026.02.0." Zugriff am: 27. April 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/reference/modules/EV/EvManager/autogenerated.html#evmanager>
- [22] "TIDA-010939 Referenzdesign | TI.com." Zugriff am: 29. April 2026. [Online.] Verfügbar: <https://www.ti.com/tool/de-de/TIDA-010939>
- [23] "phoenix contact CCS socket datasheet." Zugriff am: 3. Mai 2026. [Online.] Verfügbar: https://www.phoenixcontact.com/product/pdf/api/v1/MTcyMDA5OA?_realm=us&_locale=en-US&blocks=commercial-data%2Ctechnical-data%2Cdrawings%2Cclassifications%2Cenvironmental-compliance-data

- [24] DigiKey Electronics. "CCS socket | DigiKey Electronics." Zugriff am: 3. Mai 2026. [Online.] Verfügbar: <https://www.digikey.de/de/products/detail/phoenix-contact/1720098/25576714>
- [25] "foccci schematic," [Online]. Verfügbar unter: https://github.com/uhi22/foccci/blob/main/pdf/foccci_schematic_v5.2p7.pdf
- [26] ST micro. "L298 datasheet." Zugriff am: 3.5.26. [Online.] Verfügbar: www.st.com/resource/en/datasheet/l298.pdf
- [27] Texas Instruments, Incorporated [SLLA353 und A], "Isolation Glossary (Rev. A)," [Online]. Verfügbar unter: https://www.ti.com/lit/an/slla353a/slla353a.pdf?ts=1777703378173&ref_url=https%253A%252F%252Fti.com%252Flit%252Fpdf%252Fslla353
- [28] "ACS712-Datasheet," [Online]. Verfügbar unter: <https://www.allegromicro.com/-/media/files/datasheets/acs712-datasheet.ashx>
- [29] Texas Instruments, Incorporated [SBOSAF5 und B], "TMCS1126-Q1 AEC-Q100, Precision 500kHz Hall-Effect Current Sensor With Reinforced Working Voltage Overcurrent Detection and Ambient Field Rejection datasheet (Rev. B)," [Online]. Verfügbar unter: <https://www.ti.com/lit/ds/symlink/tmcs1126-q1.pdf?ts=1777645502209>
- [30] Texas Instruments, Incorporated [SBAS740 und B], "ADS1118-Q1 Automotive, Low-Power, SPI-Compatible, 16-Bit Analog-to-Digital Converter With Internal Reference and Temperature Sensor datasheet (Rev. B)," [Online]. Verfügbar unter: <https://www.ti.com/lit/ds/symlink/ads1118-q1.pdf?ts=1777717695847>
- [31] Texas Instruments, Incorporated [SLLSFK3 und *], "ISOW774x-Q1 Quad-Channel Digital Isolator with Integrated Low-Emissions, Low-Noise DC-DC Converter datasheet," [Online]. Verfügbar unter: <https://www.ti.com/lit/ds/symlink/isow7741-q1.pdf?ts=1777707377926>
- [32] "Protron DC Leitungsschutzschalter, LS-Schalter, MCB 4-polig 2-polig, Sicherungsautomaten für Solar PV Photovoltaikanlagen 2P, 4P, 600V 1000V 10-63A, polaritätsunabhängig | plentyShop LTS." Zugriff am: 4. Mai 2026. [Online.] Verfügbar: https://www.arcs-europe.de/haustechnik/solartechnik/zubehoer/protron-dc-leitungsschutzschalter-ls-schalter-mcb-sicherungsautomaten-fuer-solar-pv-photovoltaikanlagen-2p-4p_1291_2772
- [33] "Installation Guide," *EVerest Core*. Zugriff am: 4. Mai 2026. [Online.] Verfügbar: <https://mintlify.wiki/EVerest/everest-core/installation#prerequisites>
- [34] Electronic Systems Design. "PCB high voltage spacing: What every engineer should know - Electronic Systems Design." Zugriff am: 7. Mai 2026. [Online.] Verfügbar: <https://blogs.sw.siemens.com/electronic-systems-design/2025/04/29/pcb-high-voltage-spacing-what-every-engineer-should-know/>

- [35] Wikipedia. "Consistent Overhead Byte Stuffing." Zugriff am: 10. Mai 2026. [Online.] Verfügbar: https://en.wikipedia.org/w/index.php?title=Consistent_Overhead_Byte_Stuffing&oldid=1344930661
- [36] GitHub. "EVerest: YetiEvDriver." Zugriff am: 10. Mai 2026. [Online.] Verfügbar: <https://github.com/EVerest/EVerest/tree/main/modules/HardwareDrivers/EV/YetiEvDriver>
- [37] GitHub. "EVerest - EvManager source." Zugriff am: 11. Mai 2026. [Online.] Verfügbar: <https://github.com/EVerest/EVerest/tree/main/modules/EV/EvManager>
- [38] "ev_board_support - EVerest Manual 2026.02.0." Zugriff am: 11. Mai 2026. [Online.] Verfügbar: https://everest.github.io/nightly/reference/interfaces/ev_board_support.html
- [39] P. P. "Raspberry Pi cross development." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://ttapa.github.io/Pages/Raspberry-Pi/>
- [40] GitHub. "ttroy50/cmake-examples: Useful CMake Examples." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://github.com/ttroy50/cmake-examples>
- [41] GitHub. "ttapa/docker-arm-cross-toolchain: Modern GCC cross-compilation toolchains for Raspberry Pi." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://github.com/ttapa/docker-arm-cross-toolchain>
- [42] Wikipedia. "CMake." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://de.wikipedia.org/w/index.php?title=CMake&oldid=262443251>
- [43] YouTube. "How to CMake Good - Recommended Order." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://www.youtube.com/playlist?list=PLK6MXr8gasrGmliSuVQXpfFuE1uPT615s>
- [44] "Development — Charge Control C User Guide." Zugriff am: 31. Mai 2026. [Online.] Verfügbar: https://chargebyte-docs.readthedocs.io/projects/everest-charge-control-c/en/everest_charge_control_c_1.6.0/development.html
- [45] YouTube. "Grazer Linuxtage 2021 - Yocto Crashkurs." Zugriff am: 1. Juni 2026. [Online.] Verfügbar: <https://www.youtube.com/watch?v=8z9DpS5rIUA>
- [46] "Linux / Yocto - EVerest Manual 2026.02.0." Zugriff am: 31. Mai 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/explanation/linux-yocto/index.html>
- [47] "Yocto Linux Integration," *EVerest Core*. Zugriff am: 31. Mai 2026. [Online.] Verfügbar: <https://mintlify.wiki/EVerest/everest-core/deployment/yocto>
- [48] "ev-cli - EVerest Manual 2026.02.0." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/explanation/dev-tools/ev-cli.html#ev-cli>

- [49] GitHub. "EVerest/everest-admin-panel." Zugriff am: 29. April 2026. [Online.] Verfügbar: <https://github.com/EVerest/everest-admin-panel>
- [50] "Get started in Software - EVerest Manual 2026.02.0." Zugriff am: 18. Mai 2026. [Online.] Verfügbar: <https://everest.github.io/nightly/how-to-guides/getting-started/get-started-sw.html>
- [51] GitHub. "dspace-group/dsV2Gshark: Free Wireshark plugin to analyze ISO15118 V2G communication between electric vehicles (EV) and charging stations (EVSE)." Zugriff am: 19. Mai 2026. [Online.] Verfügbar: <https://github.com/dspace-group/dsV2Gshark>
- [52] Wikipedia. "Wireshark." Zugriff am: 19. Mai 2026. [Online.] Verfügbar: <https://de.wikipedia.org/w/index.php?title=Wireshark&oldid=257457503>
- [53] PlatformIO. "PlatformIO: Your Gateway to Embedded Software Development Excellence." Zugriff am: 19. Mai 2026. [Online.] Verfügbar: <https://platformio.org/>
- [54] Wikipedia. "PlatformIO." Zugriff am: 19. Mai 2026. [Online.] Verfügbar: <https://de.wikipedia.org/w/index.php?title=PlatformIO&oldid=256749101>
- [55] GitHub. "dspace-group/dsV2Gshark: Free Wireshark plugin to analyze ISO15118 V2G communication between electric vehicles (EV) and charging stations (EVSE)." Zugriff am: 1. Juni 2026. [Online.] Verfügbar: https://github.com/dspace-group/dsV2Gshark/tree/main/Wireshark/dsV2Gshark_examples

Anhang

Anhang A	Laufzeitkonfiguration	50
Anhang B	Schaltplan Dieter	52
Anhang C	Platine Dieter	53
Anhang D	Yocto	54

Anhang A Laufzeitkonfiguration

Laufzeitkonfiguration für eine DC Ladesimulation. Der Ablauf wird mit `auto_exec_commands` festgelegt

```
settings:
  telemetry_enabled: true
active_modules:
  DieterEvDriver_instance:
    module: DieterEvDriver
    connections: {}
    config_module:
      baud_rate: 19200
      serial_port: /dev/ttyUSB0
  EvManager0:
    module: EvManager
    connections:
      ev:
        - module_id: PyEvJosev0
          implementation_id: ev
      ev_board_support:
        - module_id: DieterEvDriver_instance
          implementation_id: board_support
      kvs:
        - module_id: Store0
          implementation_id: main
      slac:
        - module_id: EvSlac0
          implementation_id: main
    config_module:
      ac_nominal_voltage: 230
      auto_enable: true
      auto_exec: true
      auto_exec_commands: 'sleep 2;iso_wait_pwm_is_run-
ning;iso_wait_slac_matched;iso_start_v2g_session
DC;iso_wait_pwr_ready;iso_dc_power_on;iso_wait_for_stop 120;unplug'
      auto_exec_infinite: false
      connector_id: 1
      dc_discharge_max_current_limit: 300
      dc_discharge_max_power_limit: 150000
      dc_discharge_target_current: 5
      dc_discharge_v2g_minimal_soc: 20
      dc_energy_capacity: 60000
      dc_max_current_limit: 300
      dc_max_power_limit: 150000
      dc_max_voltage_limit: 900
      dc_target_current: 5
      dc_target_voltage: 200
      departure_time: 86400
      e_amount: 0
      force_payment_option: false
      keep_cross_boot_plugin_state: false
      max_current: 16
      plugin_commands: ''
      soc: 30
      support_sae_j2847: false
      three_phases: false
  PyEvJosev0:
    module: PyEvJosev
    connections: {}
    config_module:
      device: eth0
      enable_tls_1_3: false
      enforce_tls: false
      is_cert_install_needed: false
      is_internet_service_needed: false
      request_all_service_details: false
```

```
select_all_vas_services: false
supported_DIN70121: false
supported_ISO15118_2: true
supported_ISO15118_20_AC: false
supported_ISO15118_20_DC: true
supported_d20_energy_services: DC,DC_BPT
tls_active: false
Store0:
  module: Store
  connections: {}
EvSlac0:
  module: EvSlac
  connections: {}
  config_implementation:
    main:
      device: eth0
      set_key_timeout_ms: 500
```

Sheet: /
 File: Dieter_EvController.kicad.sch
 Title: Dieter controller for EV side
 Size: A4
 Date: 20.4.
 KiCad E.D.A. 10.0.0
 Rev: 1
 Id: 1/1

52

Anhang C Platine Dieter

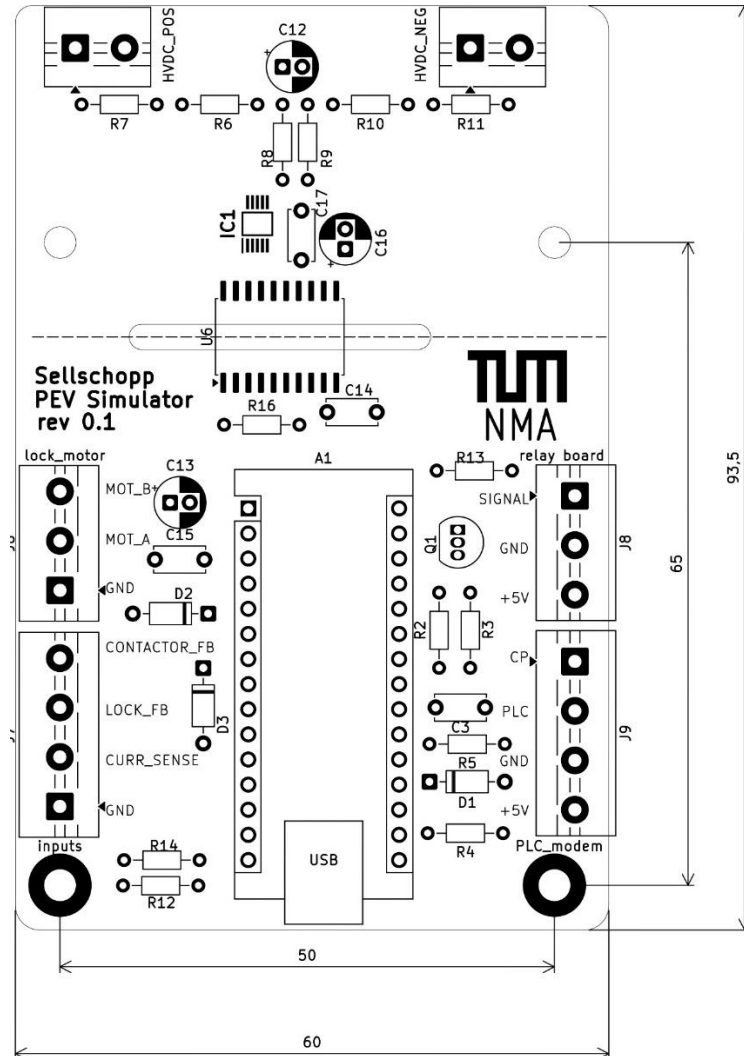


Abbildung C.1 Maße Platine

Anhang D Yocto

conf/local.conf:

```
1. MACHINE = "raspberrypi3-64"
2.
3. # Parallelization (adjust to your CPU)
4. BB_NUMBER_THREADS = "10"
5. PARALLEL_MAKE = "-j 10"
6.
7. # Disk space monitoring
8. BB_DISKMON_DIRS = "STOPTASKS,{TMPDIR},1G,100K STOPTASKS,{DL_DIR},1G,100K"
9.
10. # Add Everest to image
11. IMAGE_INSTALL:append = " everest-core mosquitto python3 kernel-module-cdc-acm kernel-module-
ch341 kernel-module-usbserial dieter-ev-driver python3-iso15118 openjdk-17-jre tshark"
12.
13.
14. ENABLE_UART = "1"
15.
16. DISTRO_FEATURES:append = " systemd usrmerge wifi"
17. INIT_MANAGER = "systemd"
18.
19. LICENSE_FLAGS_ACCEPTED = "synaptics-killswitch"
20.
21. EVEREST_CORE_PATH = "/path/to/evereste"
22. BBMASK += "meta-everest/recipes-connectivity/openssl/"
23.
24. # common download cache for all builds
25. DL_DIR = "/some/shared/directory"
26.
27. # common sstate cache (reusable build artefacts)
28. SSTATE_DIR = "/another/shared/directory"
29.
30. # root login without password
31. EXTRA_IMAGE_FEATURES += "debug-tweaks"
```